

Natural Language Processing (NLP)

Danae Sanchez, Post-doctoral Researcher, DIKU
Deep Learning, January 2026

Hola!



Large Language Models (LLMs) can generate human-like text

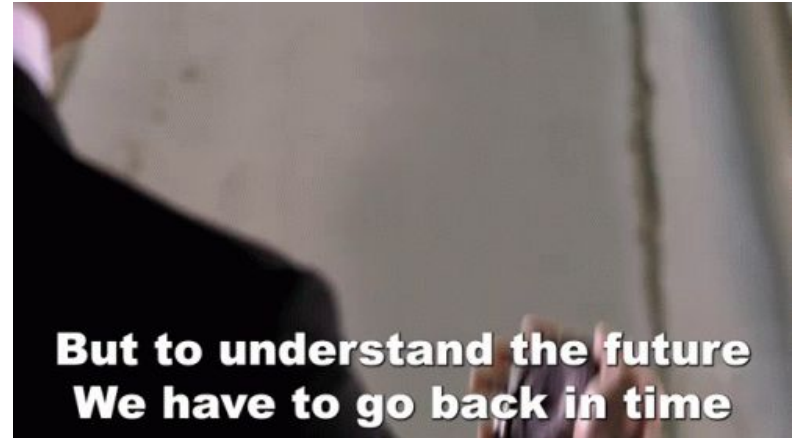
Write a list of three activities
to do in Copenhagen during
winter

Wonderful Copenhagen in winter! Here are three activities to do in this amazing city during the winter season:

1. **Visit the Tivoli Gardens Christmas Market**: Tivoli Gardens is a magical place during winter, and its Christmas market is a must-visit. The park is transformed into a winter wonderland with twinkling lights, festive decorations, and cozy stalls selling traditional Danish food, drinks, and crafts. Try some gløgg (mulled wine) and æbleskiver (Danish doughnuts) to get into the holiday spirit!
2. **Go ice skating at the City Hall Square (Rådhuspladsen)**: Every winter, the City Hall Square in Copenhagen is transformed into a large outdoor ice skating rink. It's a fun and festive activity for all ages, and you can even rent skates if you don't have your own. Afterwards, warm up with a cup of hot chocolate or coffee at one of the nearby cafes.
3. **Take a stroll through the festive Nyhavn harbor**: Nyhavn is one of Copenhagen's most iconic neighborhoods, and during winter, it's especially charming. The colorful houses are decorated with festive lights, and the harbor is filled with historic boats and ships. Take a stroll along the waterfront, visit the Christmas market stalls, and enjoy the cozy atmosphere. You might even catch a glimpse of Santa Claus himself!

I hope you enjoy your winter wonderland experience in Copenhagen!

How did we get here?

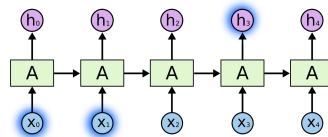


Advances in NLP

← 2017:



- BoW, Word2Vec, RNNs, Seq2seq



2017: The Transformer



2018: GPT, BERT



2019: GPT-2



2020: GPT-3

2022: ChatGPT - instruction tuning + RLHF

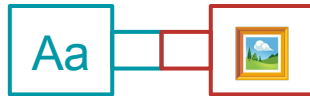
2023: GPT-4, LLaMA, Claude



Advances in NLP (continued)

2024: Open and Multimodal LLMs

- Gemini 1.5, Claude 3, GPT-4V, Qwen-VL, LLaVA 🔥
- Emergence of Mixture-of-Experts and efficient fine-tuning



2025: Agentic & reasoning models

- DeepSeek-R1 → chain-of-thought & reasoning focus
- Agentic AI systems (AutoGPT, OpenDevin, Claude Agents)
- Stronger multi-image and temporal reasoning
- Multi-image, video



2026: ?

Outline

1. Language Modelling
2. Neural Language Models
3. Transformers for Language Modelling

1. Language Modelling

Language Modelling: Predicting the next word

Language models assign probabilities to word **sequences** capturing how natural or likely they read.

$$P(W) = P(w_1, w_2, \dots, w_n)$$

$$P(\text{"Today is Saturday"}) > P(\text{"Today Saturday is"})$$

$$P(\text{"Today is Saturday"}) > P(\text{"Today is tacos"})$$

Language Modelling: Predicting the next word

Language modeling involves creating a probabilistic model to **predict the next word in a sequence** given the preceding words.

$$P(w_n | w_1, w_2, \dots, w_{n-1})$$

The cat sat on the ____

- $P(\text{'mat'})=0.6$
- $P(\text{'roof'})=0.2$
- $P(\text{'laptop'})=0.05$



Modelling an entire sequence of words

P (The cat sat on the mat)

$$P(w_1, w_2, \dots, w_n) = P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \cdots P(w_n|w_1, w_2, \dots, w_{n-1})$$

chain rule of probability

We could estimate the joint probability of an entire sequence of words by multiplying together a number of conditional probabilities.

Problem: in practice we can't actually compute the exact probability of a word given a *long sequence* of preceding words, $P(w_n|w_1:n-1)$.

N-gram Language Models

The Markov Assumption

Instead of computing the probability of a word given its entire history, we can approximate the history by just the last few words.

N-gram Models (N=2)

$$P(w_n | w_{1:n-1}) \approx P(w_n | w_{n-1})$$

Estimating Probabilities

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)}$$

count of the bigram $C(w_{n-1}w_n)$ and normalize by the sum of all the bigrams that share the same first word w_{n-1}

Example: Bigram Language Model

The cat sat on the mat

Bigram	Count	Probability
(The → cat)	50	$50 / 85 = 0.58$
(cat → sat)	30	$30 / 30 = 1.0$
(sat → on)	25	$25 / 25 = 1.0$
(on → the)	40	$40 / 40 = 1.0$
(the → mat)	35	$35 / 85 = 0.412$



$$P(\text{The cat sat on the mat}) = P(\text{cat} \mid \text{The}) \times P(\text{sat} \mid \text{cat}) \times P(\text{on} \mid \text{sat}) \times P(\text{the} \mid \text{on}) \times P(\text{mat} \mid \text{the})$$

Example: Bigram Language Model

The **tacocat** sat on the mat

Bigram	Count	Probability
(The → tacocat)	0	$0 / 85 = 0$
tacocat → sat	0	$0 / 0 !!$
(sat → on)	25	$25 / 25 = 1.0$
(on → the)	40	$40 / 40 = 1.0$
(the → mat)	35	$35 / 85 = 0.412$

$P(\text{The } \mathbf{tacocat} \text{ sat on the mat}) = 0$



Smoothing: Handling Zero Probabilities

Smoothing adjusts probabilities so unseen events get a small non-zero probability, while reducing probabilities of seen events slightly.

Add-on (Laplace) smoothing: add 1 to every count.

$$P(w_n \mid w_{n-1}) = \frac{C(w_{n-1}, w_n) + 1}{C(w_{n-1}) + |V|}$$

count of the bigram $C(w_{n-1}w_n)$ and normalize by the sum of all the bigrams that share the same first word w_{n-1}

Example: Bigram Language Model



The **tacocat** sat on the mat

Bigram	Count	Smoothed Count	Probability
(The → tacocat)	0	$0 + 1 = 1$	$1 / (85+101) = 0.005$
(tacocat → sat)	0	$0 + 1 = 1$	$1 / (85+101) = 0.010$
(sat → on)	25	$25 + 1 = 26$	$26 / (85+101) = 0.206$
(on → the)	40	$40 + 1 = 41$	$41 / (85+101) = 0.291$
(the → mat)	35	$35 + 1 = 36$	$36 / (85+101) = 0.194$

P(The **tacocat** sat on the mat) > 0

Why N-gram language models fall short?

N-gram models only look back a few words, they can't capture longer context or dependencies.

`"has amazing ____"` → {food, nightlife, art, culture}

- "Mexico has amazing food"
 - "New York has amazing nightlife"
 - "Paris has amazing art"
 - "Mexico has amazing culture"
- 0.25 - food
 - 0.25 nightlife
 - 0.25 - art
 - 0.25 culture

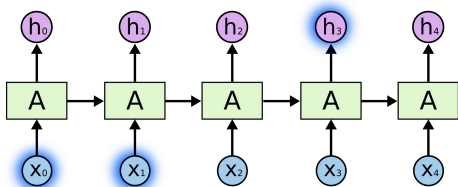
Because the model only looks at the last two words ('has amazing'), it can't distinguish between different cities — so it picks randomly among equally likely options.

2. Neural Language Models

Neural Language Models

Neural language models use neural networks to capture the dependencies between words.

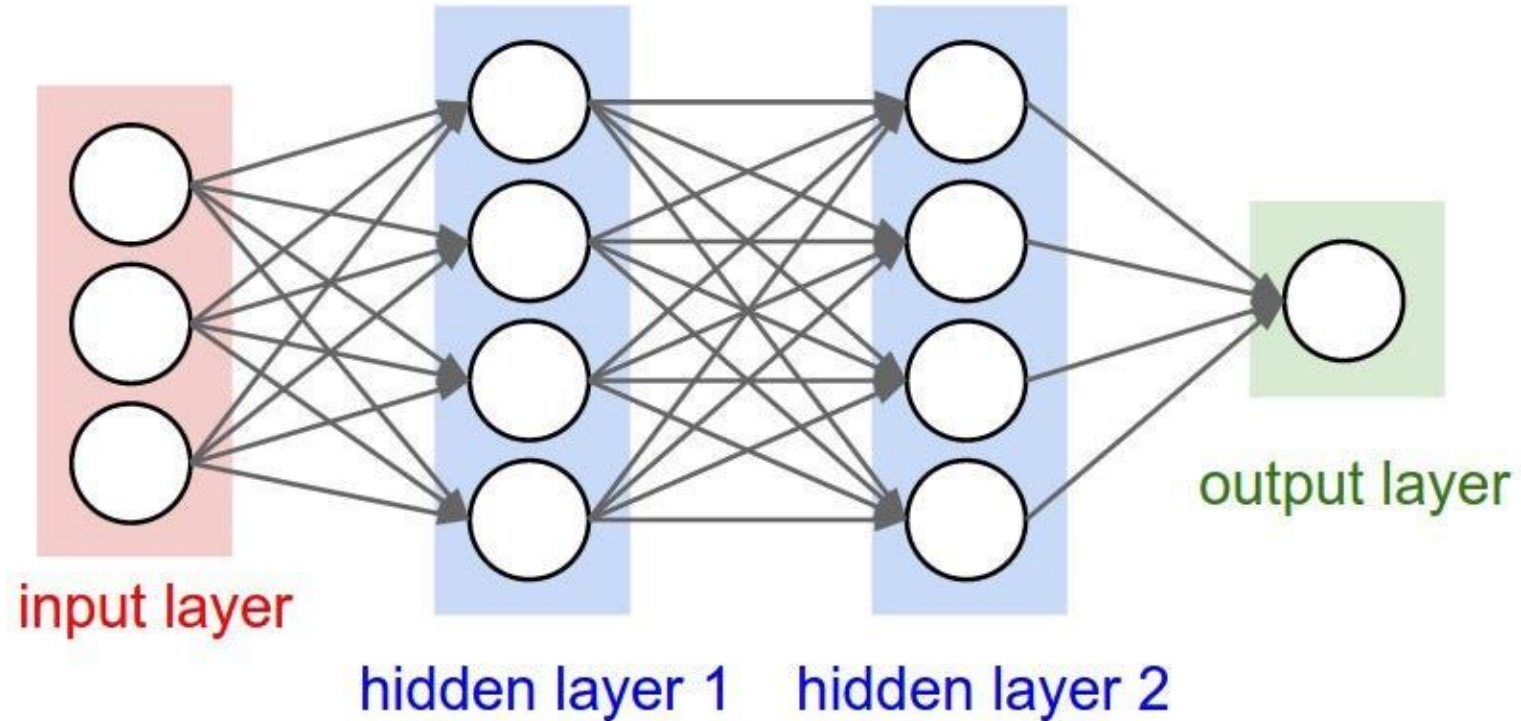
Recurrent Neural Networks (RNNs)



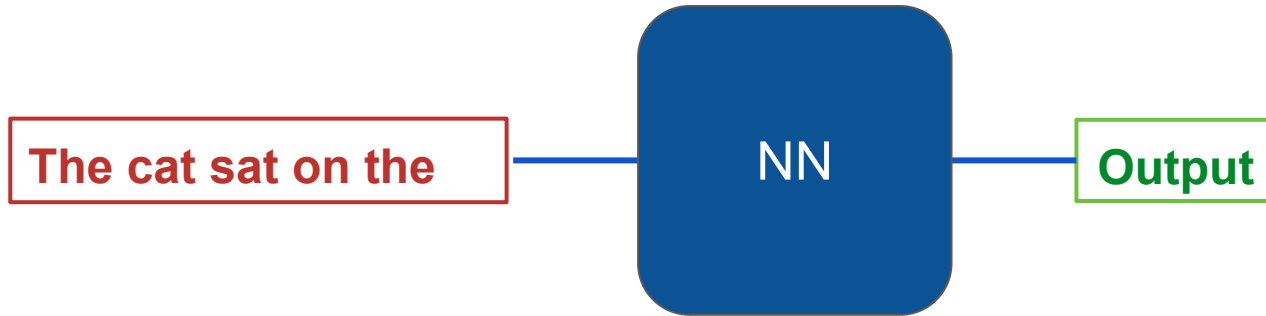
Transformers



Neural Language Models: Feed Forward Neural Network

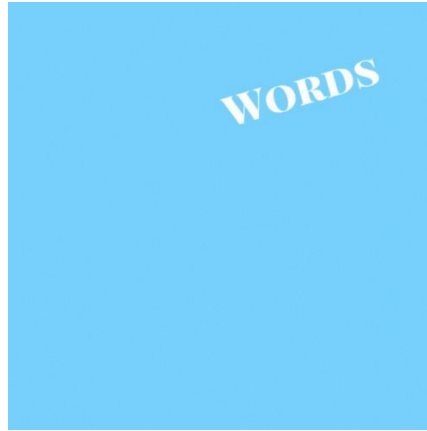


Neural Language Models



How do we feed the sentence to the NN?

How can we represent words?



Representing Text: Bag of Words

- **Term Frequency (TF) Representations**
- Each word is a **single-value** feature.

Tom chased Jerry. Jerry escaped.

Word	Count
Tom	1
chased	1
Jerry	2
escaped	1

Bag of Words turns text into **word frequency vectors**.
it captures which words appear, but
not their order or relationships.

Vector: [1,1,2,1]

Order matters!



Sentence A:

“Tom chased Jerry, and Jerry escaped.”

Sentence B:

“Jerry chased Tom, and Tom escaped.” ❌
(very different meaning!)

Bag of Words (identical):

[Tom:1, Jerry:2, chased:1, escaped:1]

Word Embeddings: Learning Meaning from Context

Tejuino



Tejuino is a cold fermented beverage made from corn and popularly consumed in the Mexican states of Jalisco, Colima, Nayarit and Oaxaca. Tejuino is usually made from corn dough, the same kind used for tortillas and tamales. The dough is mixed with water and piloncillo and boiled until the liquid is very thick.

C1: A bottle of ____ is on the table.

C2: Everybody likes ____.

C3: Don't have ____ before you drive.

C4: ____ out of corn.

Word Embeddings: Learning Meaning from Context

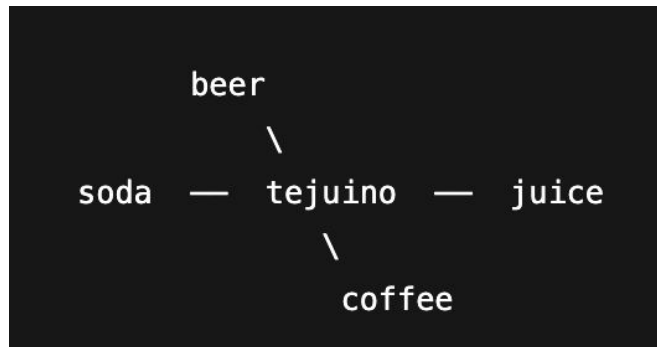
From many contexts like these:

- “A bottle of tejuino is on the table.”
- “Everybody likes tejuino.”
- “Don't have tejuino before you drive.”
- “We make tejuino out of corn.”

The model learns that “tejuino” often appears near:

drink, bottle, sweet, corn, cold, beverage

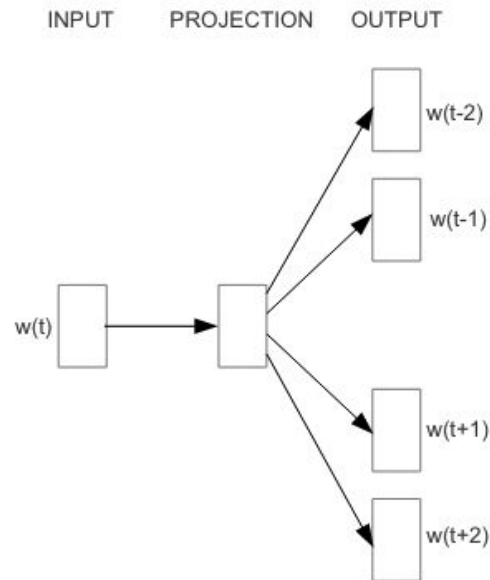
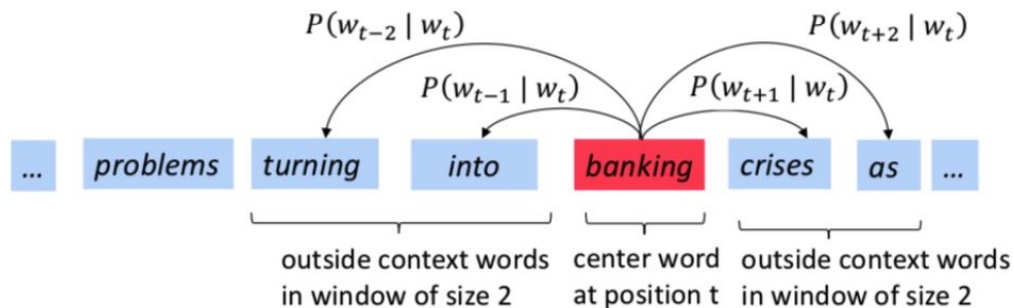
so it places “tejuino” close to “juice,” “beer,” “soda” in vector space.



Word embeddings - Representing words by its context

Skip-Gram Model:

- Predicts the context words given a target word.
- Pre-train word representation (vectors) in large corpora.



Skip-gram

Word embeddings - Representing words by its context

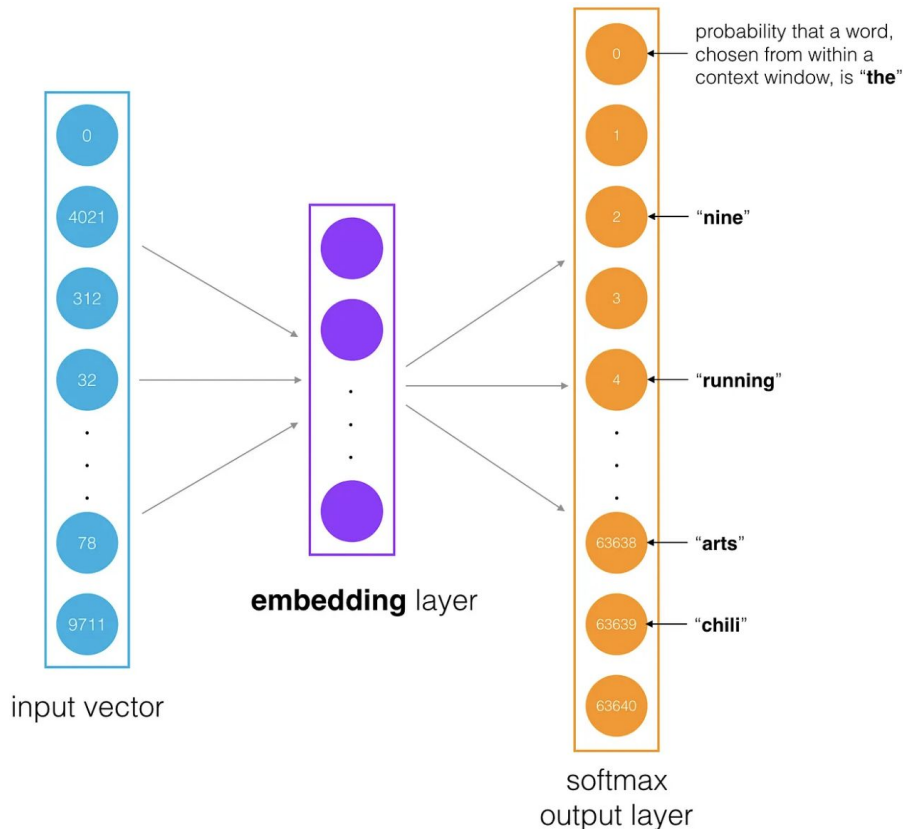
Input Layer

Size = Vocabulary size V (e.g. 10,000)

Example:

- Vocabulary: {the, cat, sat, on, mat, ..., banking}
- Input for *banking* (one-hot):

$[0, 0, 0, 0, 0, \dots, 1]$

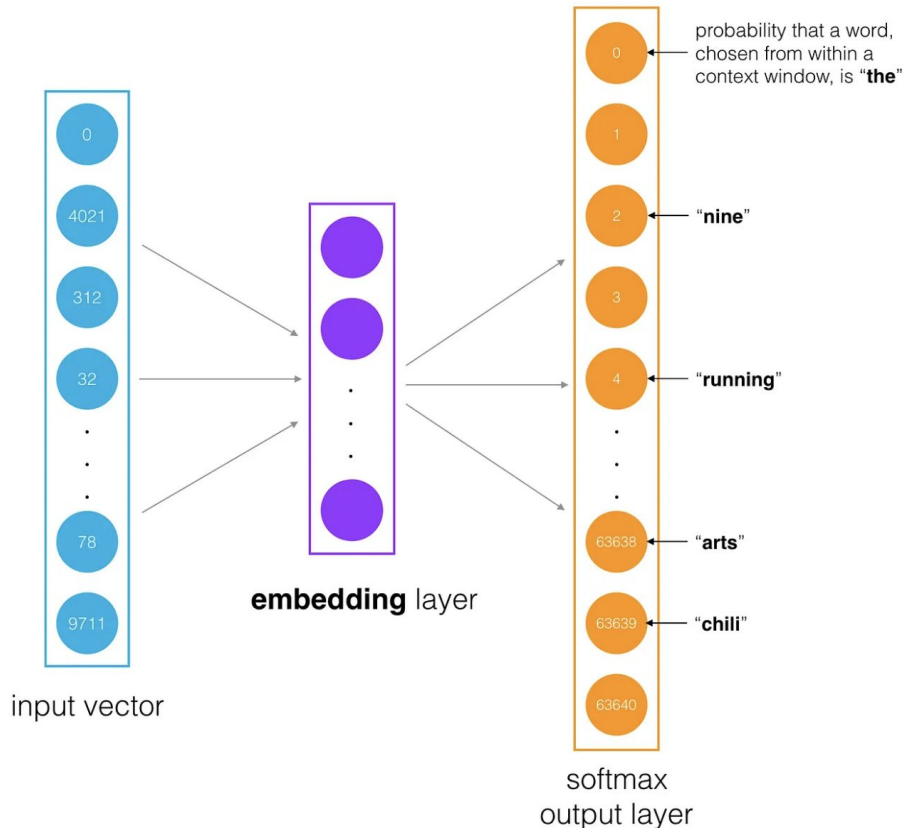


Word embeddings - Representing words by its context

Projection Layer - embedding layer

- Weight matrix $W \in \mathbb{R}^{V \times D}$
 - V : vocabulary size (e.g., 10,000)
 - D : embedding dimension (e.g. 300)
- Multiplying the one-hot vector by W **selects one row** → that row is the **embedding for the word**.

$$h = WTx$$

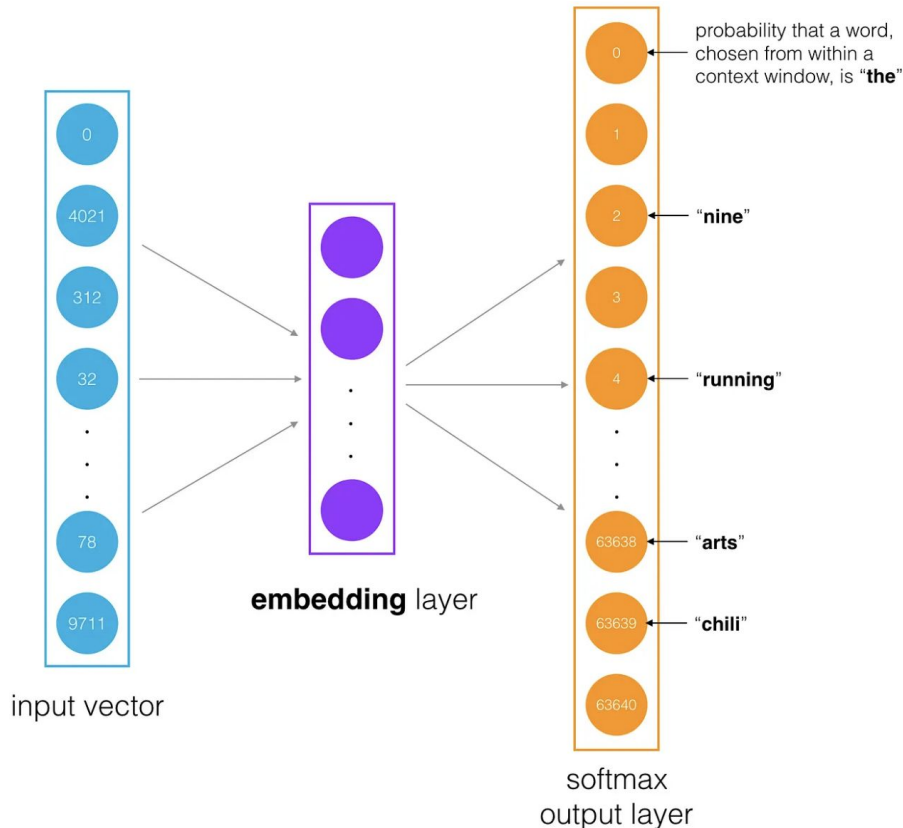


Word embeddings - Representing words by its context

Output Layer

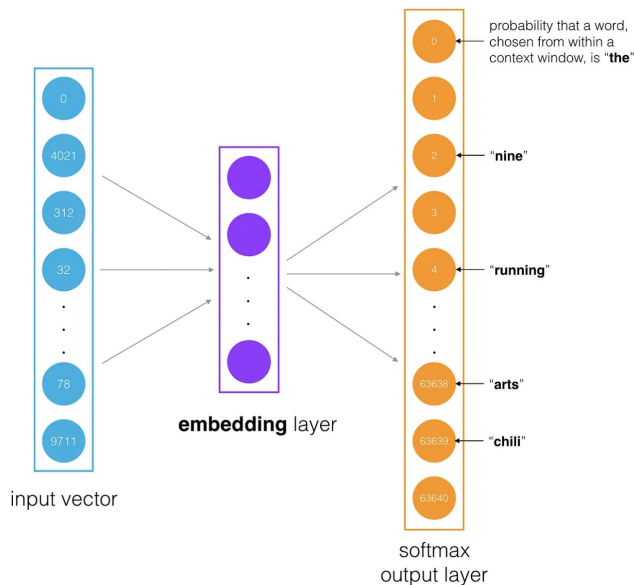
- Another matrix $W' \in \mathbb{R}^{D \times V}$
- Compute probabilities for all words using softmax:

$$P(w_{\text{context}} | w_t) = \text{softmax}(W' T h)$$



Word embeddings - Representing words by its context

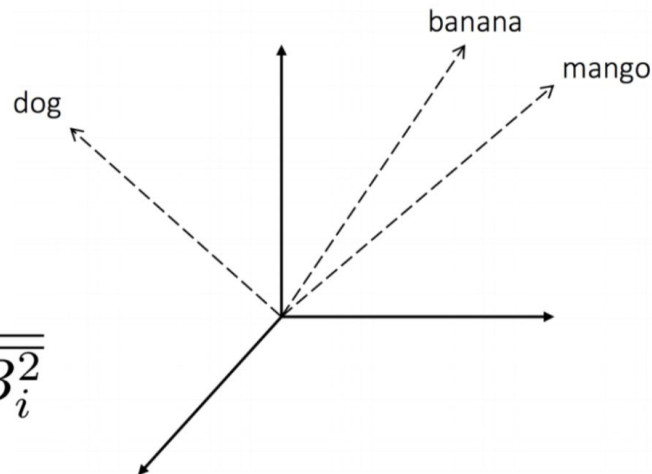
```
1 class SkipGram(nn.Module):
2     def __init__(self, n_vocab, n_embed):
3         super().__init__()
4
5         # complete this SkipGram model
6         self.embed = nn.Embedding(n_vocab, n_embed)
7         self.output = nn.Linear(n_embed, n_vocab)
8         self.log_softmax = nn.LogSoftmax(dim=1)
9
10    def forward(self, x):
11
12        # define the forward behavior
13        x = self.embed(x)
14        scores = self.output(x)
15        log_ps = self.log_softmax(scores)
16
17    return log_ps
```



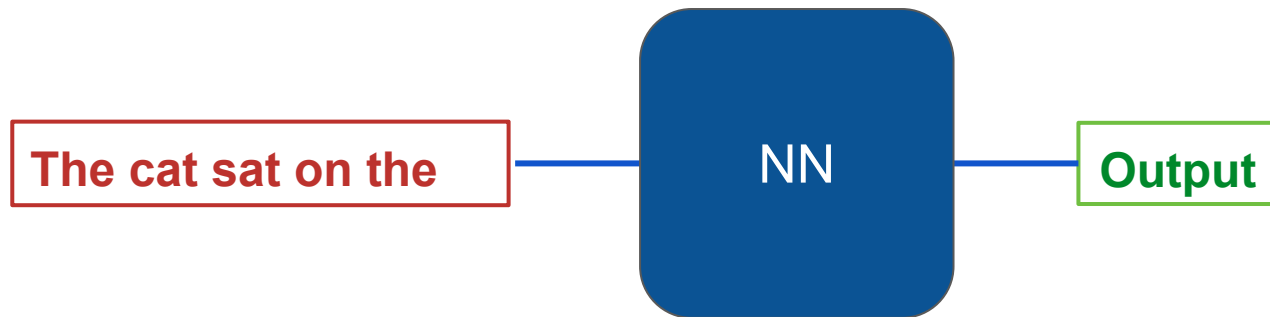
Word2Vec

- A **feature space** that **reflects** word **semantics**.
 - **Similarity**: Banana is closer to mango than dog.

$$\begin{aligned}\cos(\theta) &= \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} \\ &= \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}\end{aligned}$$

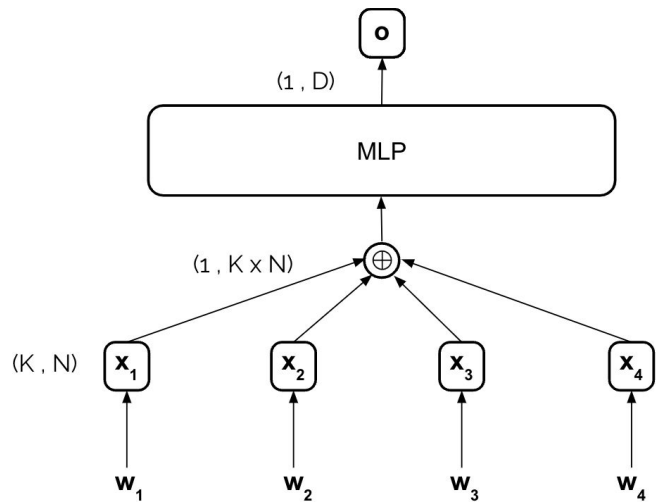


Neural Language Models

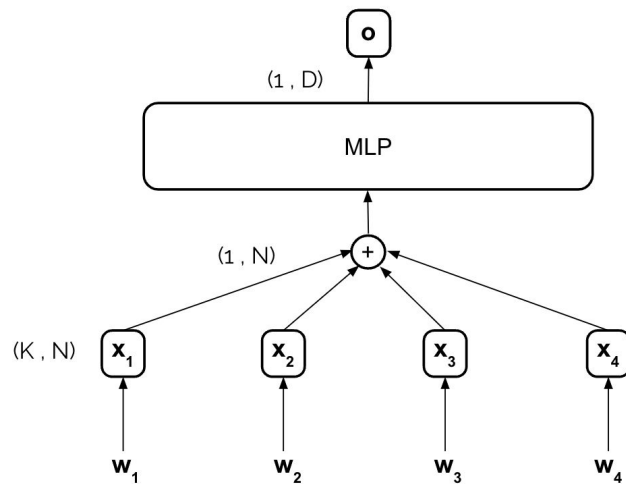


How do we feed the sentence to the NN?

Now that we can represent each word with an embedding, how do we represent a whole sentence?

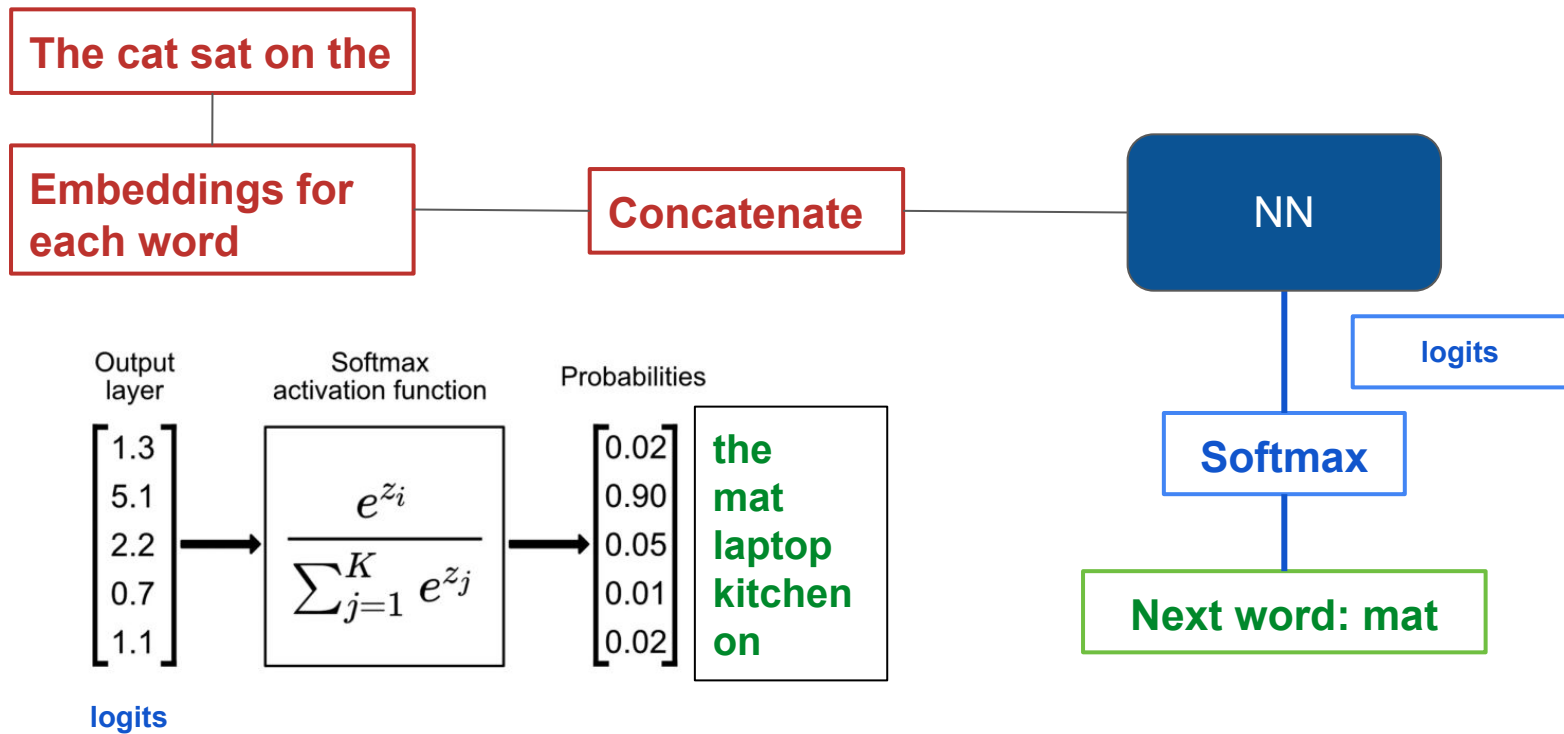


Concatenate

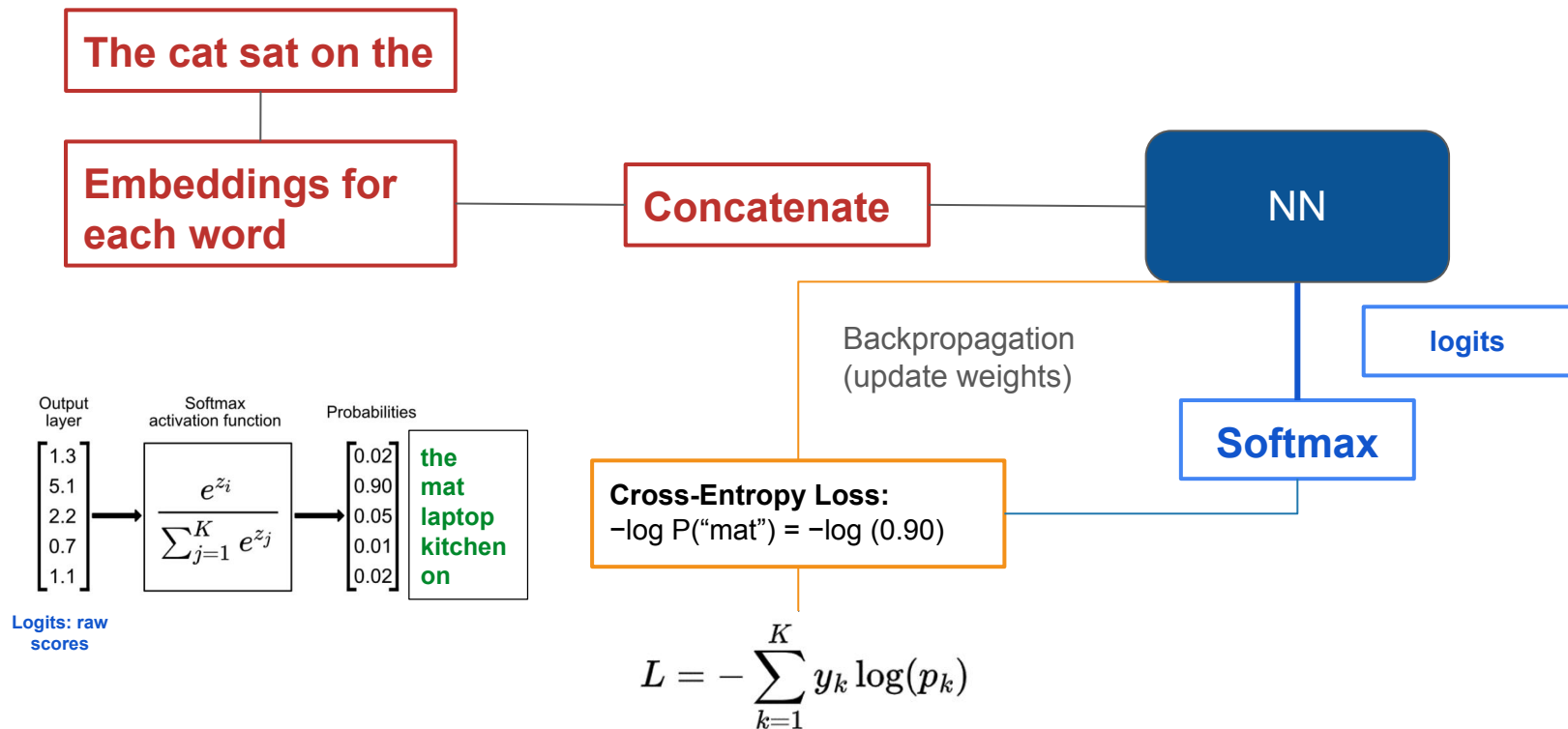


Sum

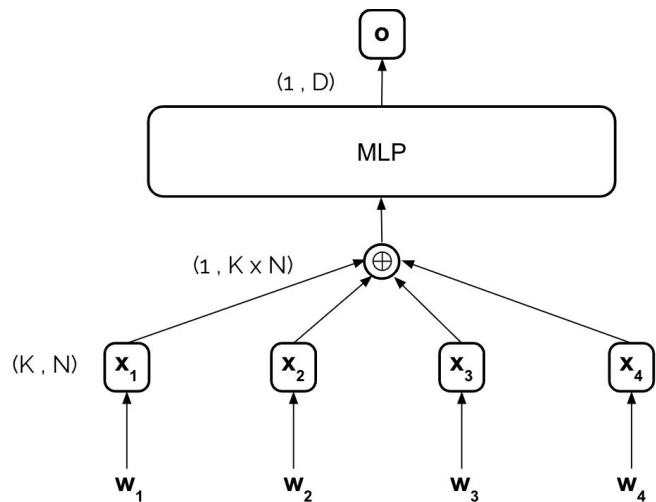
Neural Language Models



Neural Language Models: Training

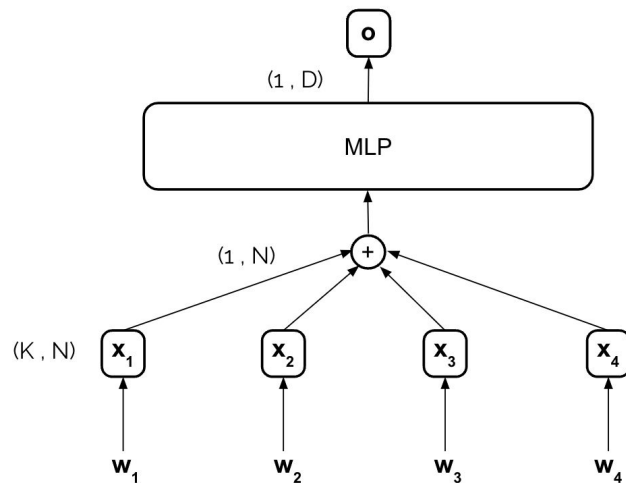


Now that we can represent each word with an embedding, how do we represent a whole sentence?



Concatenate

- It **does not scale** wrt to the #tokens.
- **Indirectly** word order matters.



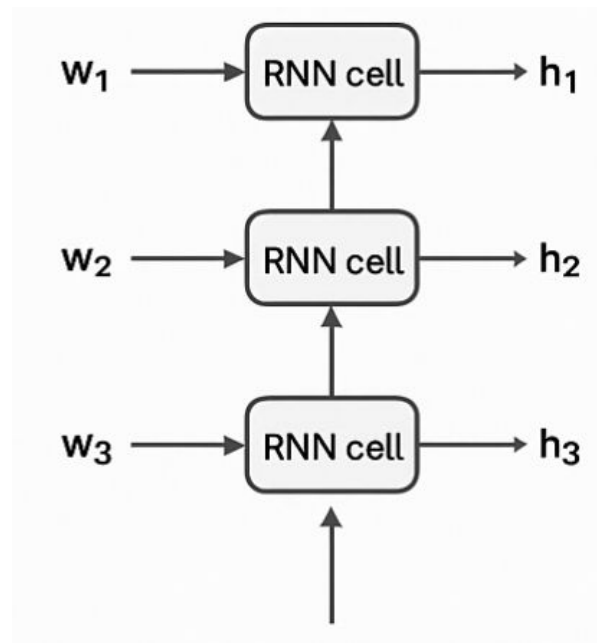
Sum

- It **scales** wrt to the #tokens.
- Word order **agnostic**.
- Extremely **Noisy** representations.

What if we could process sequences word by word, keeping track of what came before?

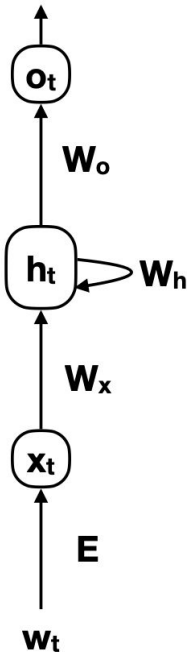
We need a model that:

- Handles variable-length input
- Remembers what came before



Simple (Vanilla) RNN (Elman, 1990)

$$\mathbf{w}_{t+1} = \text{softmax}(\mathbf{o}_t)$$



Output vector

$$\mathbf{o}_t = \mathbf{W}_o \mathbf{h}_t + \mathbf{b}_o$$

Hidden state

$$\mathbf{h}_t = \mathbf{f}(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

Embedded word

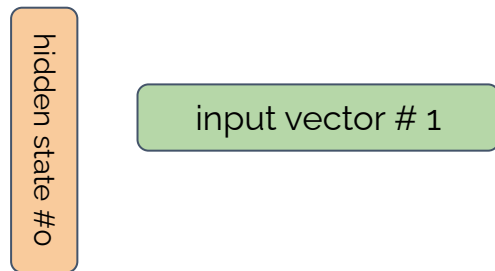
$$\mathbf{x}_t = \mathbf{E}[\mathbf{w}_t]$$

1-hot vector

$$\mathbf{w}_t = [0 \ 0 \ 0 \ 0 \ 1]$$

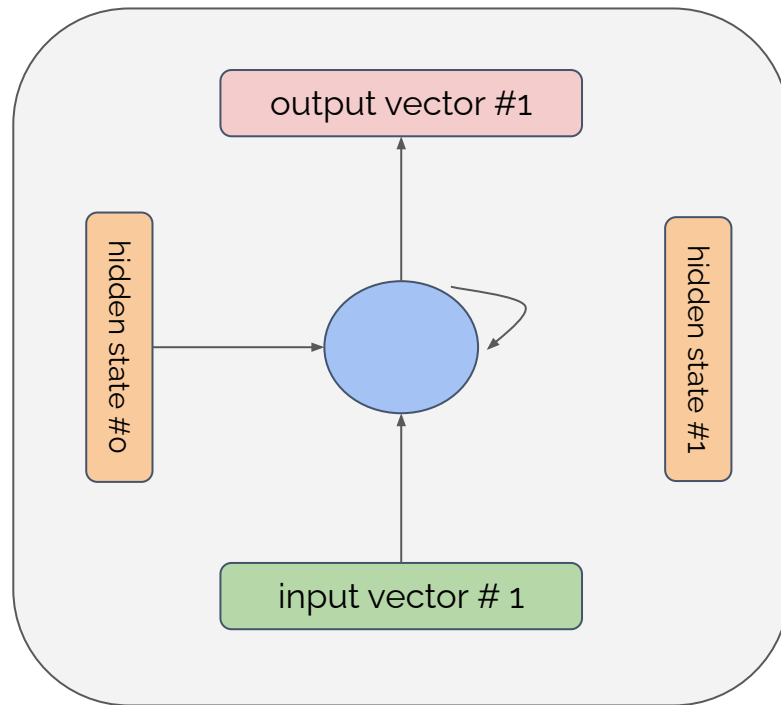
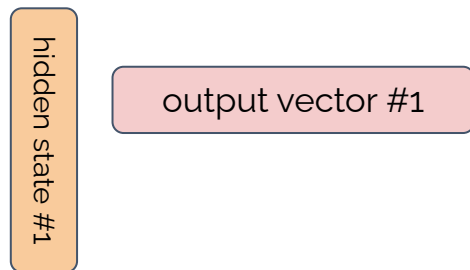
Timestep #1

Two input vectors



Process them

Two output vectors



Source: [Visualizing A Neural Machine Translation Model \(Mechanics of Seq2seq Models With Attention\)](#) – Jay Alammar

Language Modelling with RNNs

In an RNN, the hidden state $\mathbf{h}_t$ is updated based on the previous hidden state $\mathbf{h}_{t-1}$ and the current input $\mathbf{x}_t$:

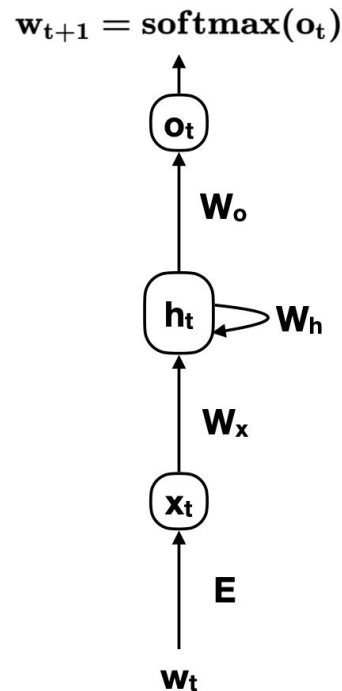
$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

where $\mathbf{f}$ is a non-linear activation function (e.g., tanh or ReLU).

The probability of the next word is then given by:

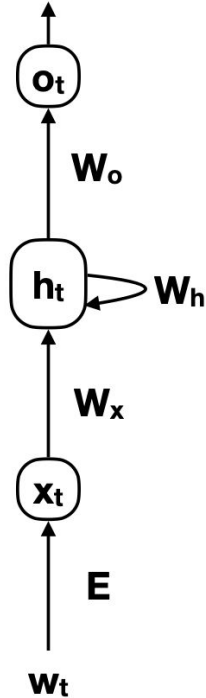
$$P(w_t | w_1, w_2, \dots, w_{t-1}) = \text{softmax}(\mathbf{W}\mathbf{h}_t + \mathbf{b})$$

where $\mathbf{W}$ and $\mathbf{b}$ are weight matrices and biases learned during training.

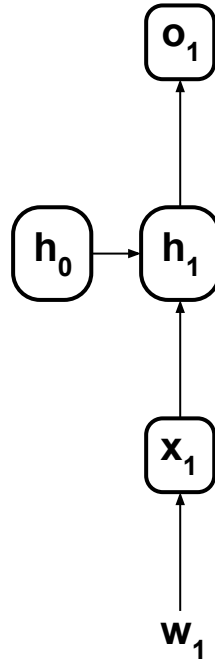


Simple (Vanilla) RNN (Elman, 1990)

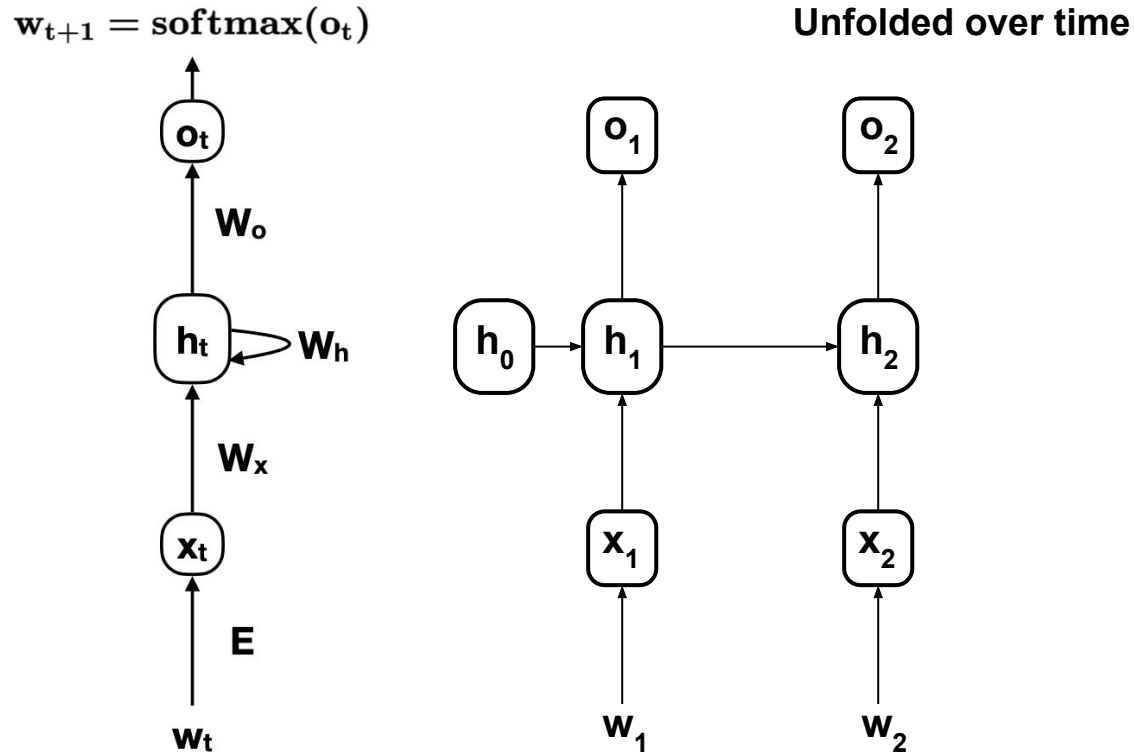
$$w_{t+1} = \text{softmax}(o_t)$$



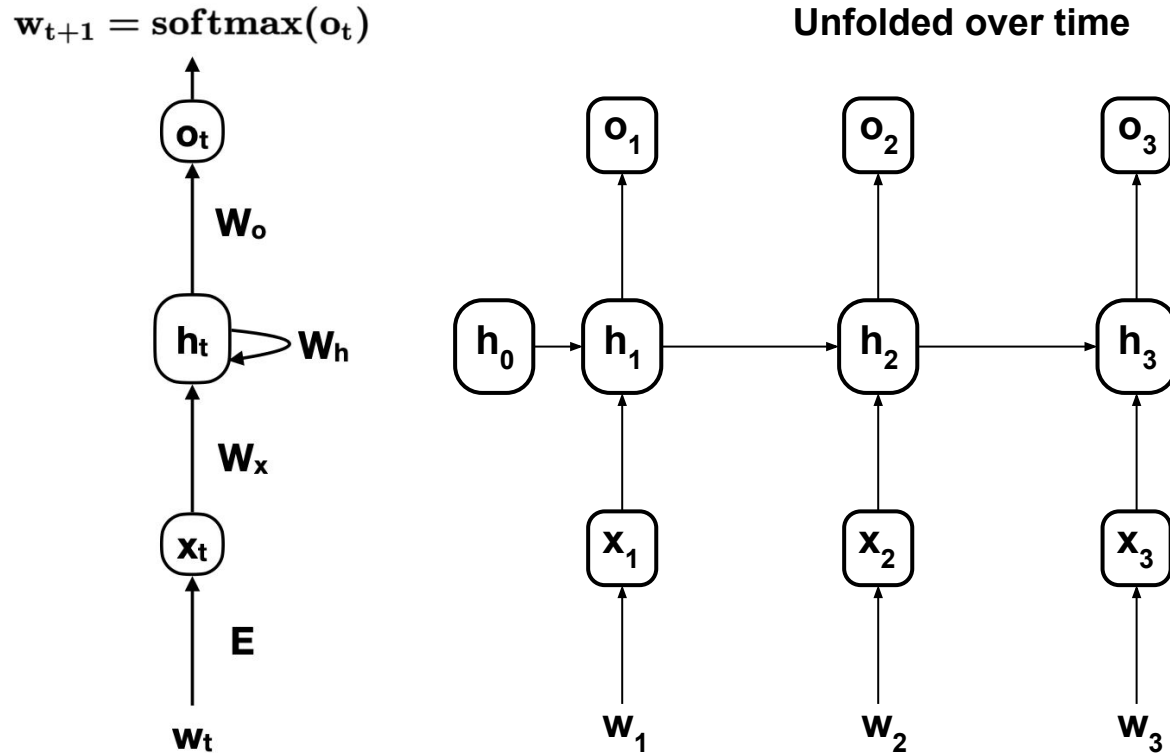
Unfolded over time



Simple (Vanilla) RNN (Elman, 1990)

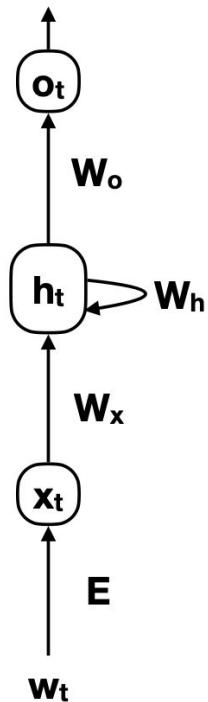


Simple (Vanilla) RNN (Elman, 1990)

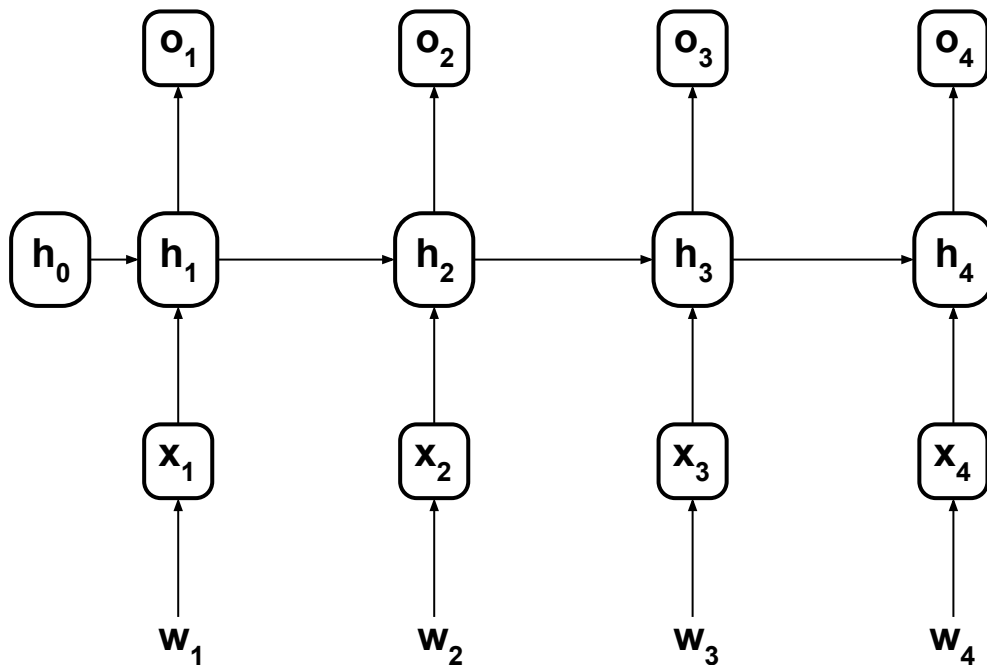


Simple (Vanilla) RNN (Elman, 1990)

$$w_{t+1} = \text{softmax}(o_t)$$



Unfolded over time



Simple (Vanilla) RNN in Torch

```
import torch
import torch.nn as nn

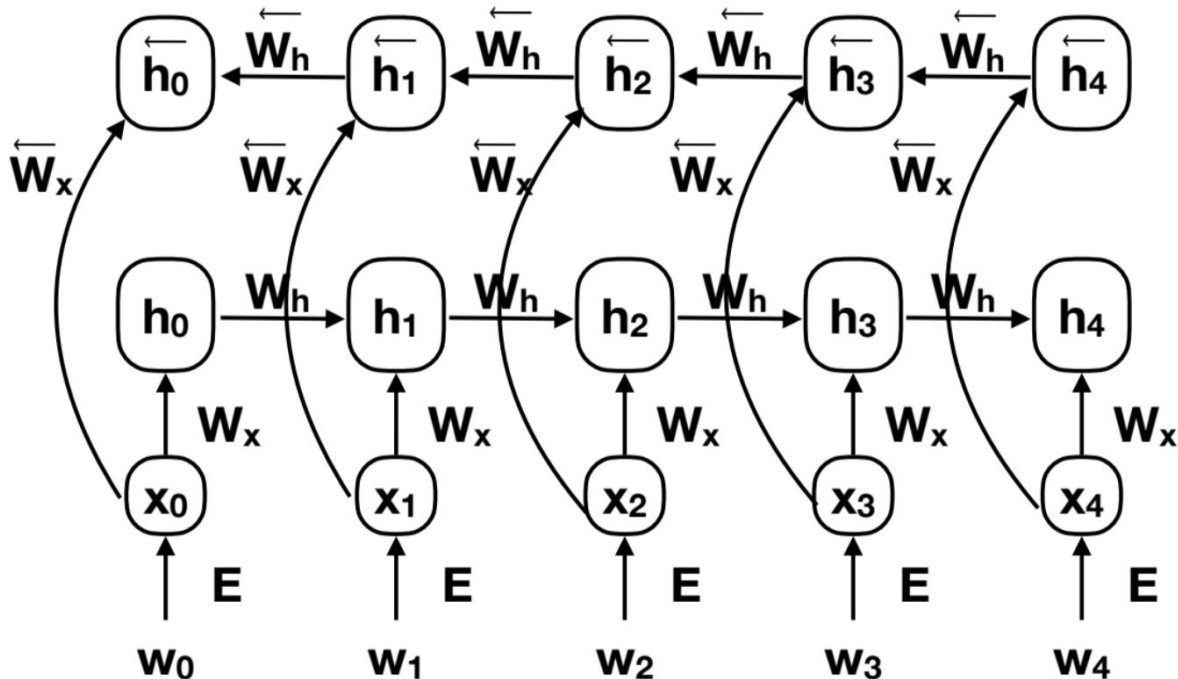
class RNNCell(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(RNNCell, self).__init__()
        self.hidden_size = hidden_size

        self.input_to_hidden = nn.Linear(input_size + hidden_size, hidden_size)
        self.hidden_to_output = nn.Linear(hidden_size, output_size)

    def forward(self, input, hidden_state):
        combined = torch.cat((input, hidden_state), 1)
        hidden_state = nn.Tanh(self.input_to_hidden(combined))
        output = self.hidden_to_output(hidden_state)
        return output, hidden_state

    def init_hidden(self):
        return torch.zeros(1, self.hidden_size)
```

Bi-directional RNNs



Backward

$$\overleftarrow{\mathbf{h}}_t = f(\overleftarrow{\mathbf{h}}_{t-1}, \mathbf{x}_t)$$

Forward

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

RNN-based Encoder in Torch

```
import torch
import torch.nn as nn

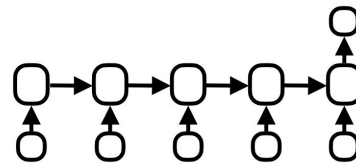
class TextRNNEncoder(nn.Module):
    def __init__(self, vocab_size, input_size, hidden_size, output_size, n_layers=2):
        super(TextRNNEncoder, self).__init__()
        self.Emb = nn.Embedding(vocab_size, input_size)
        self.RNN_f = RNN(input_size, hidden_size, output_size, n_layers)
        self.RNN_b = RNN(input_size, hidden_size, output_size, n_layers)

    def forward(self, input):
        word_embeddings = self.Emb(input)
        outputs_f = self.RNN(word_embeddings)
        outputs_b = self.RNN(torch.flip(word_embeddings, dim=(1,)))
        outputs = torch.cat((outputs_f, torch.flip(outputs_b, dim=(1,))), dim=1)

    return outputs
```

NLP Applications with RNNs

- **Classification**
 - Text Categorization
 - Named Entity Recognition



RNN-based Classifier in Torch

```
import torch
import torch.nn as nn

class TextRNNClassifier(nn.Module):
    def __init__(self, vocab_size, input_size, hidden_size, output_size,
                  n_layers=2, n_classes=2):
        super(TextRNNClassifier, self).__init__()
        self.Emb = nn.Embedding(vocab_size, input_size)
        self.RNN_f = RNN(input_size, hidden_size, output_size, n_layers)
        self.RNN_b = RNN(input_size, hidden_size, output_size, n_layers)
        self.classifier = nn.Linear(hidden_size, n_classes)

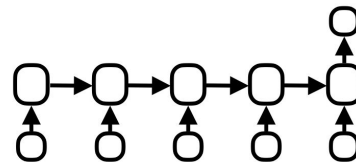
    def forward(self, input):
        word_embeddings = self.Emb(input)
        outputs_f = self.RNN(word_embeddings)
        outputs_b = self.RNN(torch.flip(word_embeddings, dim=(1,)))
        outputs = torch.cat((outputs_f[:, -1, -1, :], outputs_b[:, -1, -1, :]), dim=1)
        logits = self.classifier(outputs.squeeze(1))

    return outputs
```

NLP Applications with RNNs

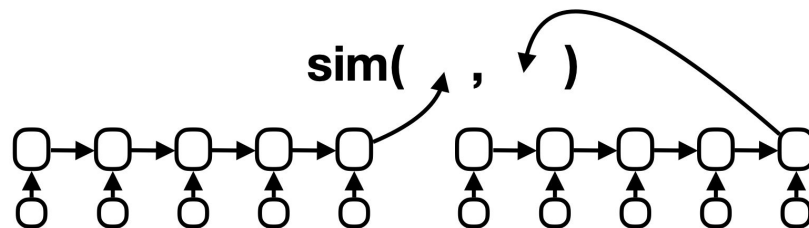
- **Classification**

- Text Categorization
- Named Entity Recognition



- **Sequence encoding**

- Sentence Similarity



RNN-based Similarity in Torch

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class TextRNNSimilarity(nn.Module):
    def __init__(self, vocab_size, input_size, hidden_size, output_size,
                  n_layers=2):
        super(TextRNNSimilarity, self).__init__()
        self.rnn_encoder = TextRNNEncoder(vocab_size, input_size, hidden_size,
                                           output_size, n_layers)

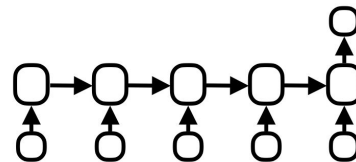
    def forward(self, input_s, input_t):
        output_s = self.rnn_encoder(input_s)[: , -1, -1, :]
        output_t = self.rnn_encoder(input_t)[: , -1, -1, :]
        sim_score = F.cosine_similarity(input_s, input_t)

        return sim_score
```

NLP Applications with RNNs

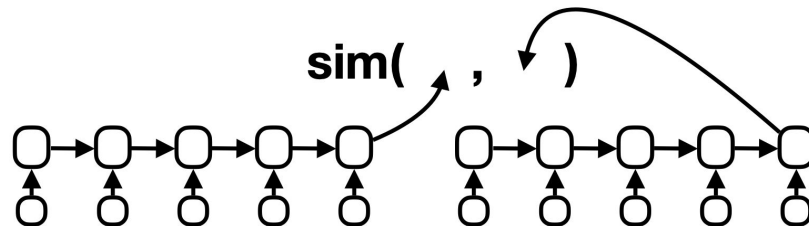
- **Classification**

- Text Categorization
- Named Entity Recognition



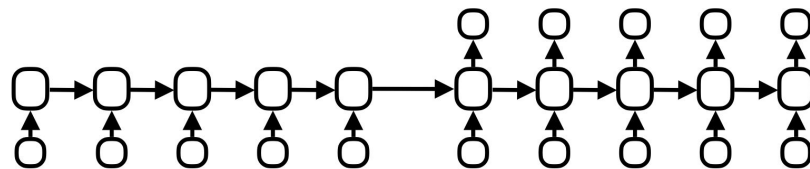
- **Sequence encoding**

- Sentence Similarity

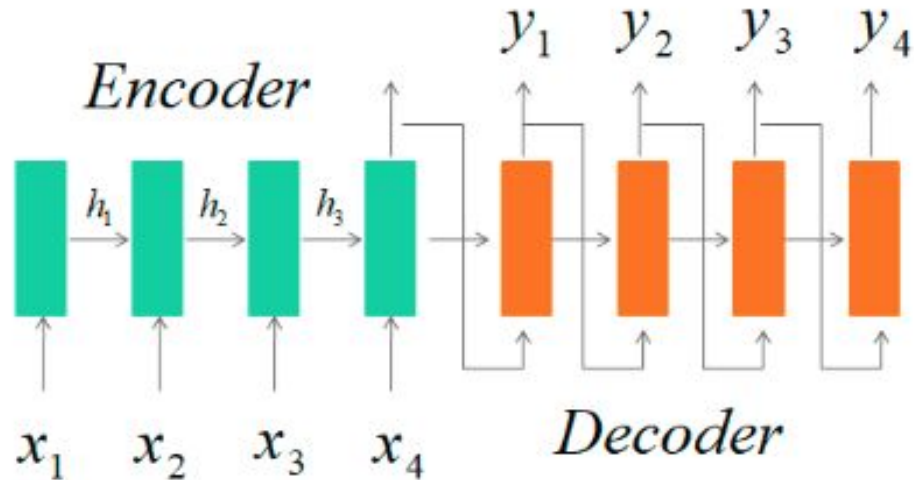
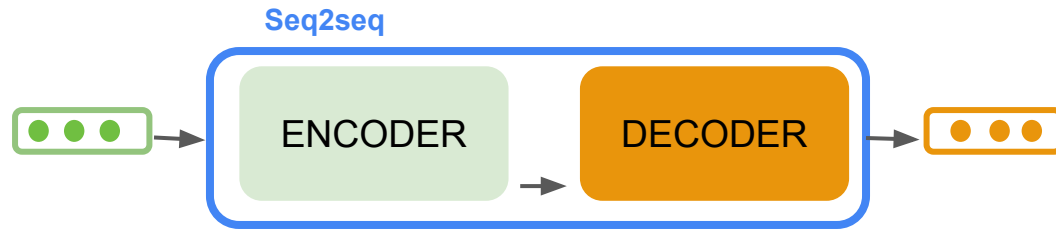


- **Sequence Generation**

- Machine Translation



Sequence to Sequence Models (Seq2seq)



```

import torch
import torch.nn as nn
import torch.nn.functional as F

class TextRNNGenerator(nn.Module):
    def __init__(self, vocab_size, input_size, hidden_size, output_size,
                  n_layers=2):
        super(TextRNNGenerator, self).__init__()
        self.rnn_encoder = TextRNNEncoder(vocab_size, input_size, hidden_size,
                                           output_size, n_layers)
        self.rnn_decoder = TextRNNDecoder(vocab_size, input_size, hidden_size,
                                           output_size, n_layers)

        self.word_decoder = nn.Linear(hidden_size, vocab_size)

    def forward(self, input):
        encoded_outputs = self.rnn_encoder(input)[: , -1, -1, :]
        decoder_inputs = torch.zeros(1, 1)
        for _ in range(self.max_gen_length):
            decoder_outputs = self.rnn_decoder(decoder_inputs, hidden=encoded_outputs)[: , -1, -1, :]
            predicted_logits = self.word_decoder(decoder_outputs)
            predicted_words = predicted_logits.argmax(1).unsqueeze(0)
            decoder_inputs = torch.cat((decoder_inputs, predicted_words), dim=1)

        return decoder_inputs

```

Limitations of RNN Language Models

Recap: What did we want to solve?

Limitation of n-gram models: only short context

Goal: Capture long-range dependencies

Limitations of RNN Language Models

Recap: What did we want to solve?

Limitation of n-gram models: only short context

Goal: Capture long-range dependencies

Have we solved it?

Partially

RNNs extend the context, but struggle with long dependencies.

Limitations of RNN Language Models

Recap: What did we want to solve?

Limitation of n-gram models: only short context

Have we solved it?

Goal: Capture long-range dependencies
Partially

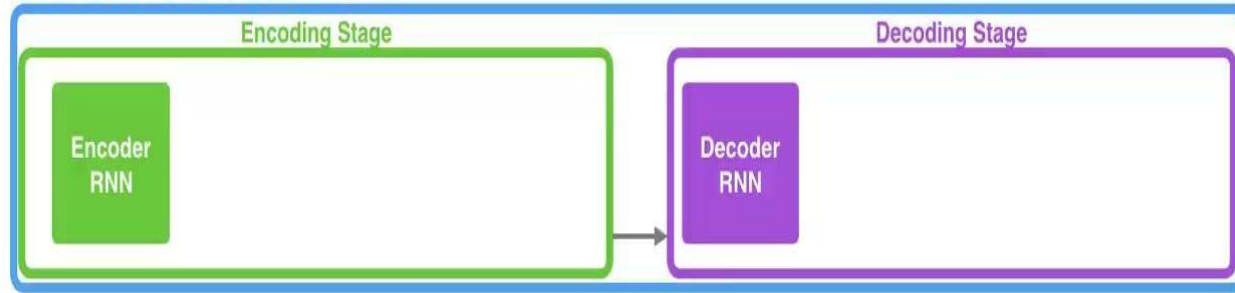
RNNs extend the context, but struggle with long dependencies.

Information fades over time

As the sequence grows, early information (e.g., the 1st token) becomes harder to retrieve by the Nth step, the hidden state becomes a noisy, diluted summary of everything seen so far.

Seq2seq: Neural Machine Translation

Neural Machine Translation SEQUENCE TO SEQUENCE MODEL



Je

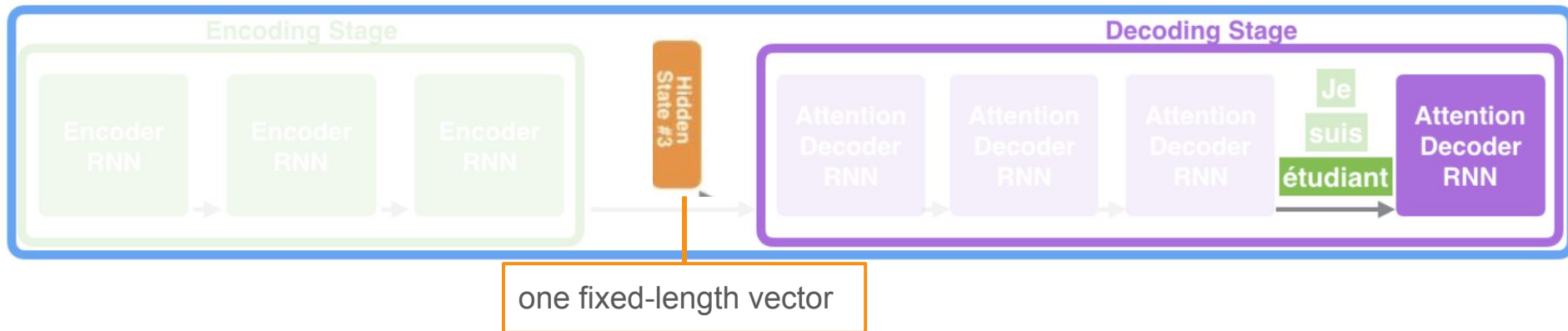
suis

étudiant

Sequence to Sequence Models (Seq2seq)

Neural Machine Translation

SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



“...the performance of a basic encoder–decoder deteriorates rapidly as the length of an input sentence increases” [Cho et al. \(2014b\)](#)

Sequence to Sequence Models (Seq2seq)

Neural Machine Translation

SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



Instead of compressing everything into one vector, the decoder can now look back at all encoder states, weighted by attention scores.

Sequence to Sequence Models (Seq2seq)

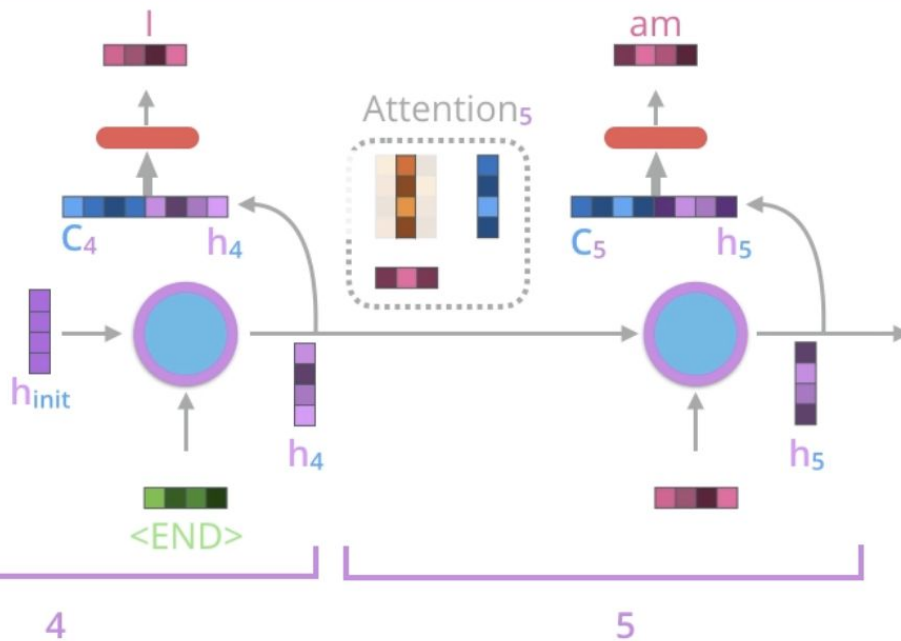
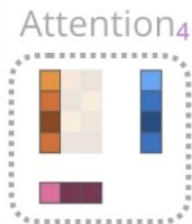
Encoding Stage



$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j.$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}$$

Attention Decoding Stage



Sequence to Sequence Models (Seq2seq)

1. The attention decoder RNN takes in the embedding of the $\langle \text{END} \rangle$ token, and an **initial decoder hidden state**.
2. The RNN processes its inputs, producing an output and a **new hidden state** vector (h_4). The output is discarded.
3. Attention Step: We use the **encoder hidden states** and the h_4 vector to calculate a context vector (C_4) for this time step.
4. We concatenate h_4 and C_4 into one vector.
5. We pass this vector through a **feedforward neural network** (one trained jointly with the model).
6. The **output** of the feedforward neural networks indicates the output word of this time step.
7. Repeat for the next time steps

Sequence to Sequence Models (Seq2seq)

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j.$$

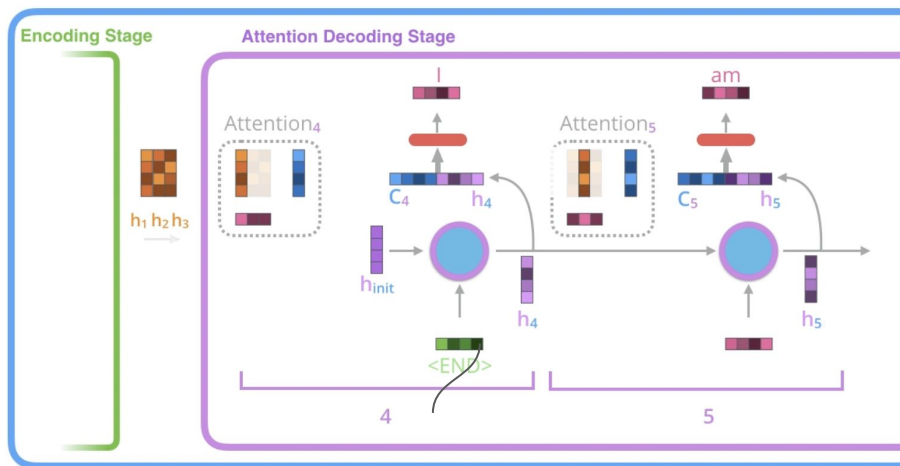
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}$$

$$e_{ij} = a(s_{i-1}, h_j)$$

" $a(.,.)$ is an alignment model which scores how well the inputs around position j (hidden states of the encoder) and the output at position i match. The score is based on the RNN hidden state of the decoder (just before emitting the current output (s_{i-1}), and the hidden state h_j of the input sentence.

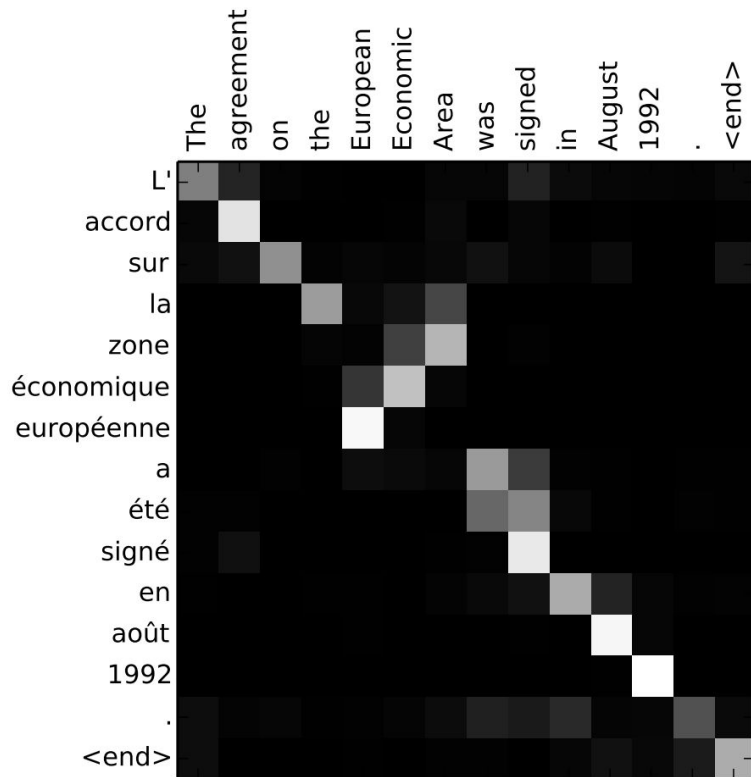
The alignment model is parametrized as a feedforward neural network which is jointly trained with the machine translation model"

Sequence to Sequence Models (Seq2seq)



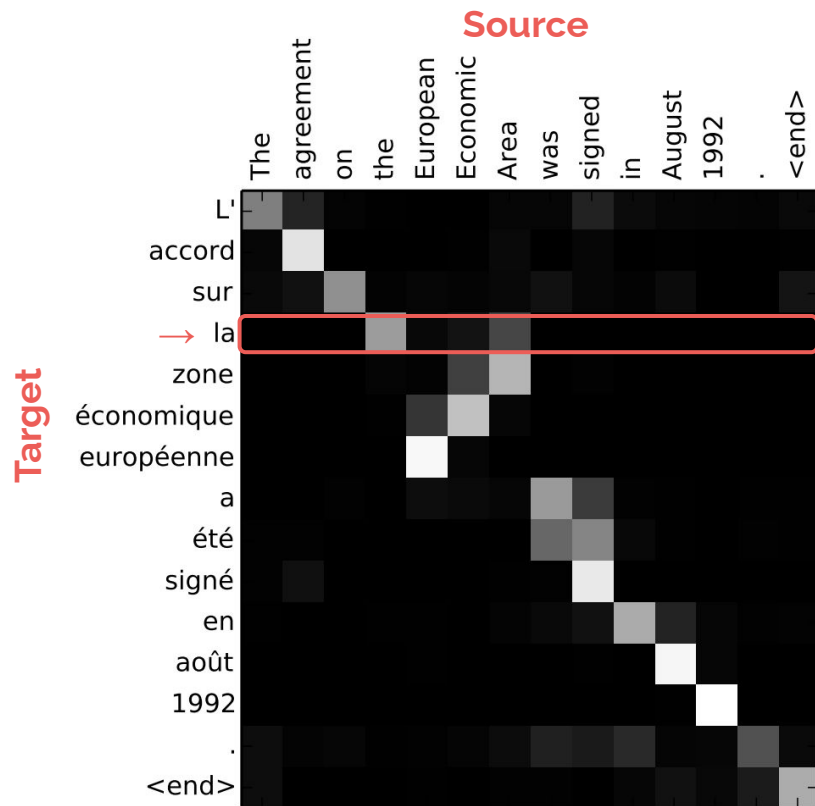
"By letting the decoder have an attention mechanism, we relieve the encoder from the burden of having to encode all information in the source sentence into a fixed length vector. With this new approach the information can be spread throughout the sequence of annotations, which can be selectively retrieved by the decoder accordingly"

Sequence to Sequence Models (Seq2seq)



The x-axis and y-axis of each plot correspond to the words in the source sentence (English) and the generated translation (French), respectively. Each pixel shows the weight a_{ij} of the annotation of the j -th source word for the i -th target word

Sequence to Sequence Models (Seq2seq)



English → French

Each cell shows how much the model focused on each input word when generating each output word.

[Neural Machine Translation by Jointly Learning to Align and Translate](#)
Bahdanau et al., 2014

Natural Language Processing (NLP)

Danae Sanchez, Post-doctoral Researcher, DIKU
Deep Learning, January 2026

Hola!



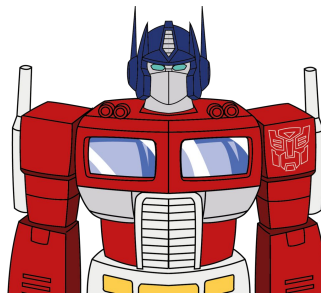
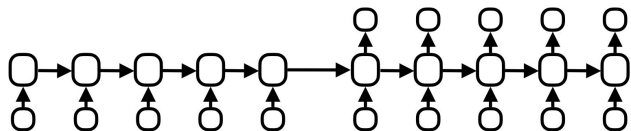
Outline

- ~~1. Language Modelling~~
- ~~2. Neural Language Models~~
3. Transformers for Language Modelling

From RNN+Attention to Transformers

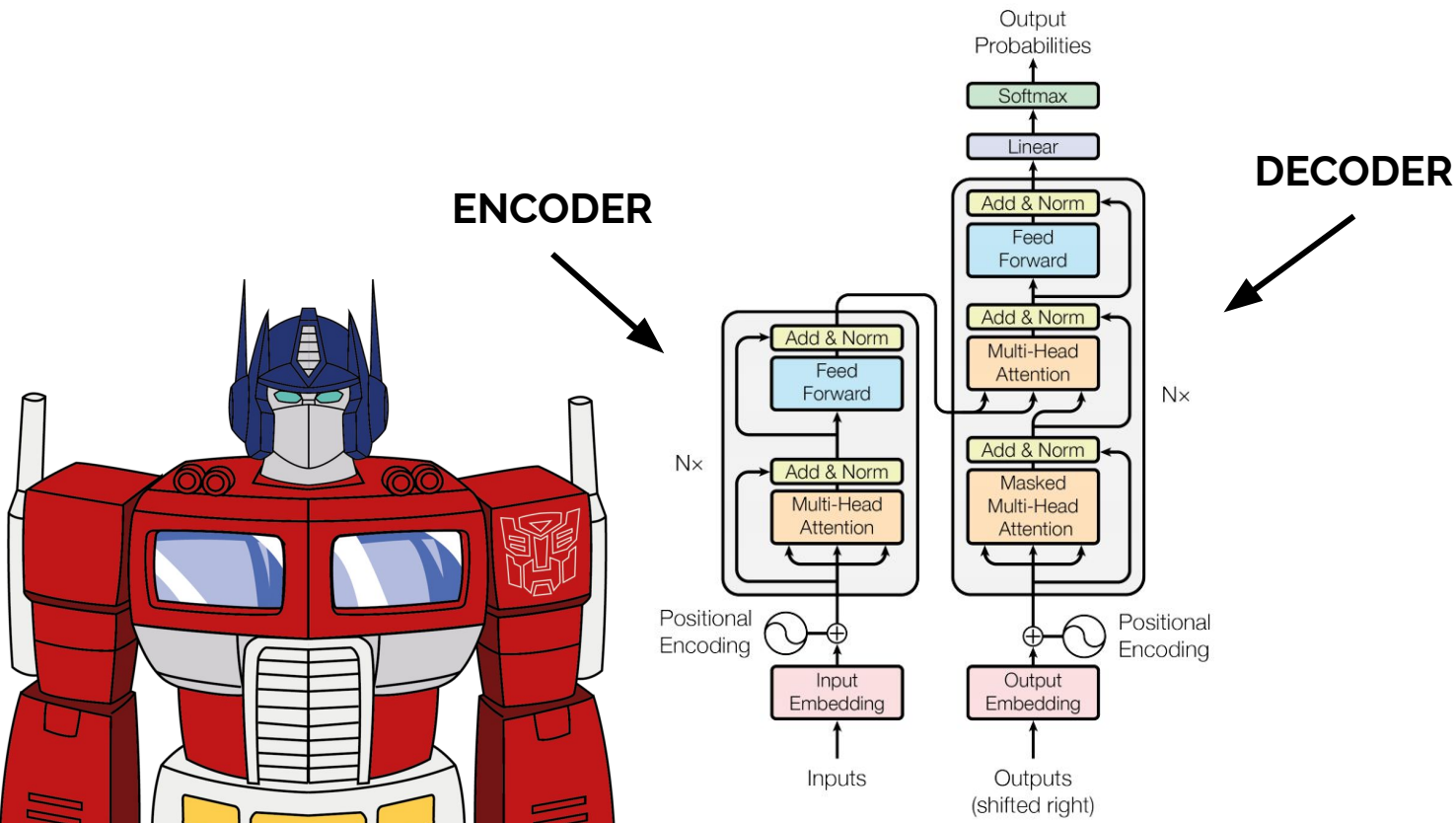
Seq2Seq + Attention $\rightarrow$ Transformer (Self-Attention everywhere)

Transformers remove recurrence and rely *entirely* on attention to model relationships between tokens.

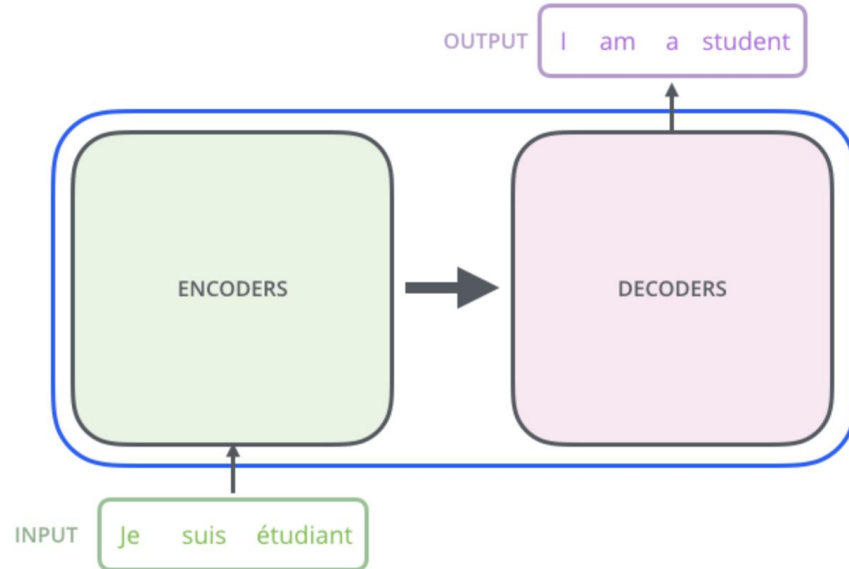


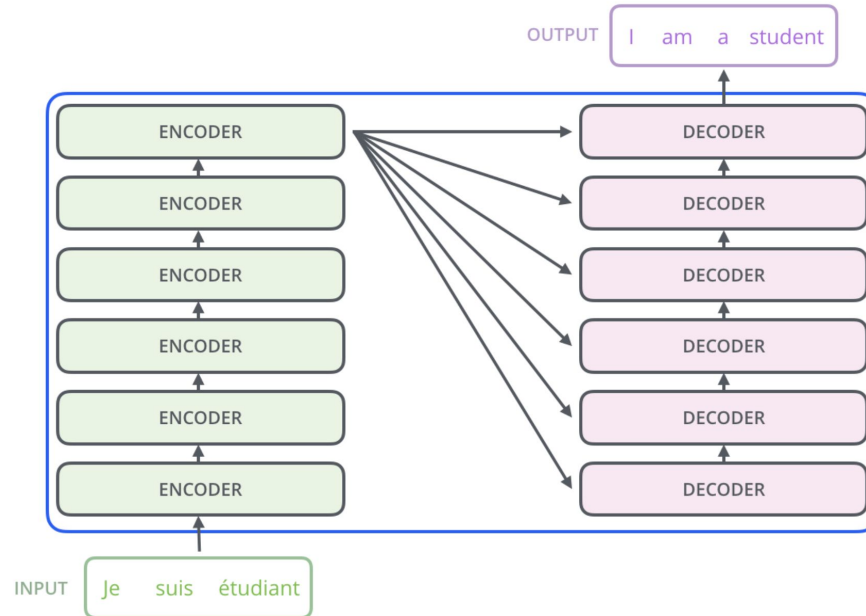
3. Transformers for Language Modelling

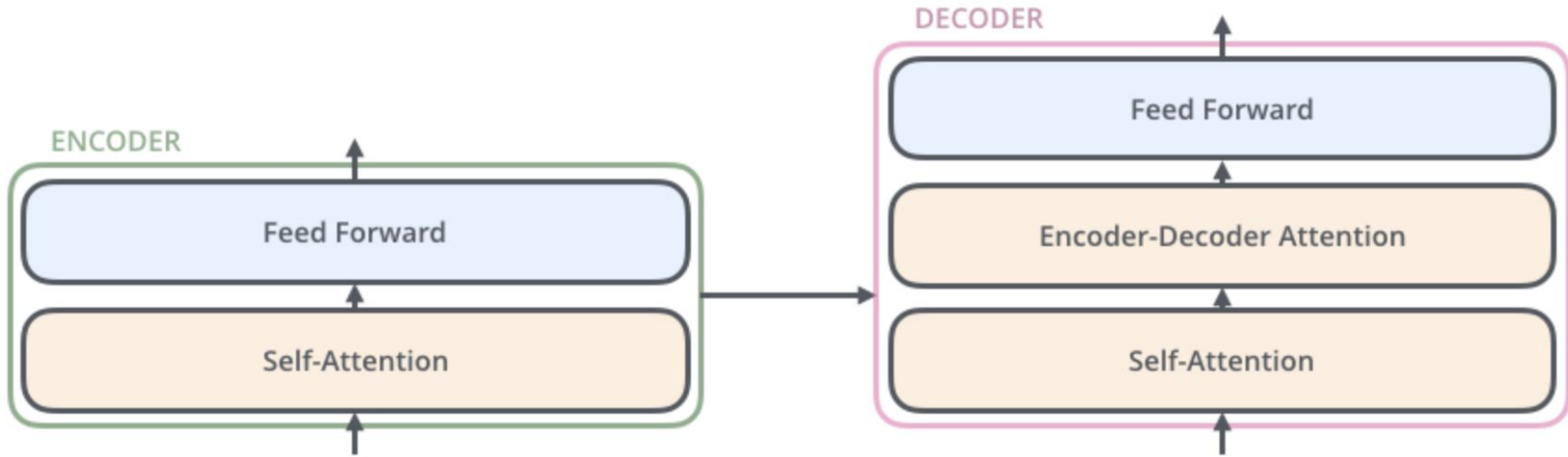
Transformers (Vaswani et al., 2017)





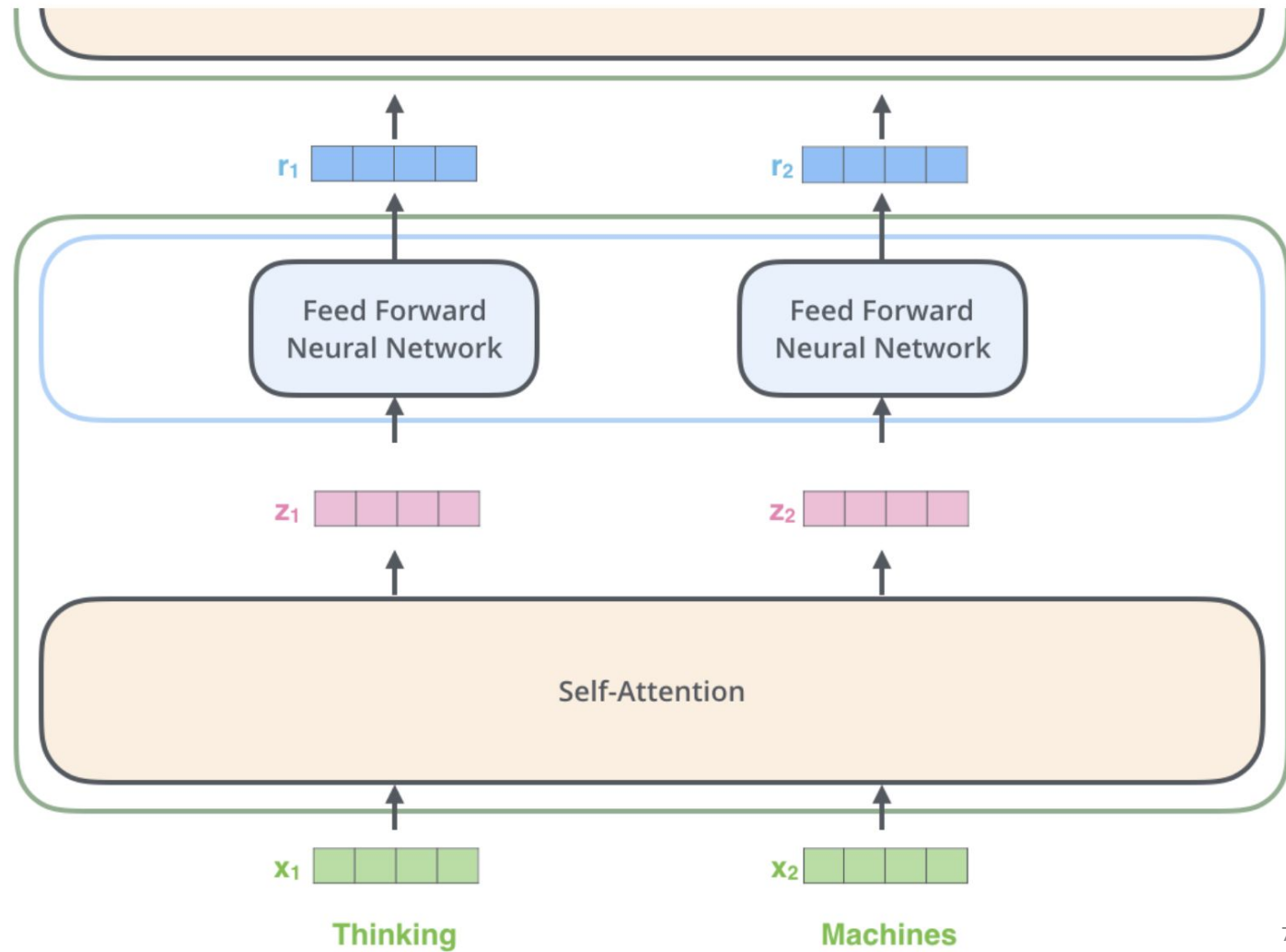






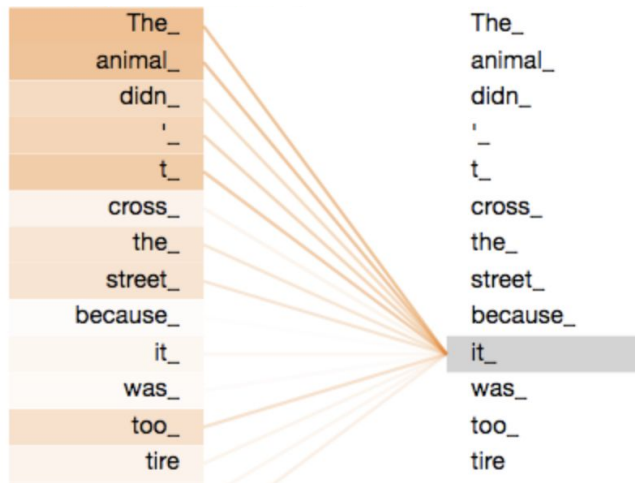
ENCODER #2

ENCODER #1



What Makes It “Self”-Attention?

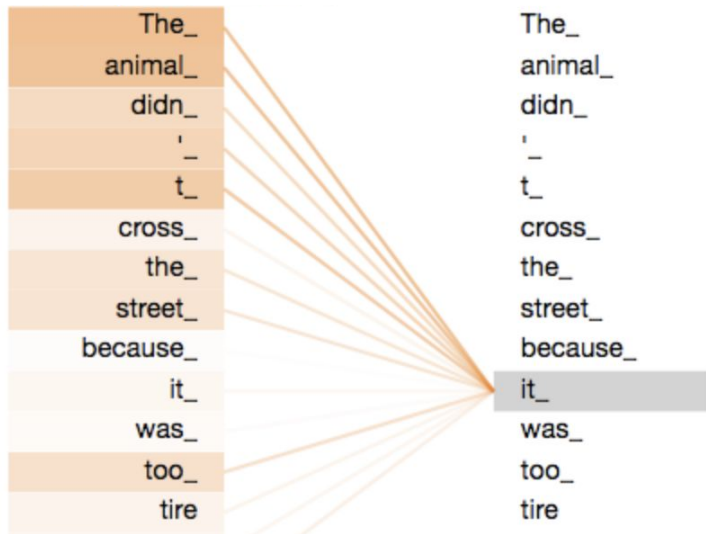
- In Seq2Seq with attention, the decoder attends to encoder outputs.
- In Transformers, each token attends to every other token in the same sequence.
- This allows parallel computation and long-range context capture.



Self-Attention

Transformers use self-attention to handle **long-range dependencies**.

"The animal didn't cross the street because **it** was too tired"



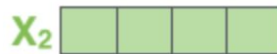
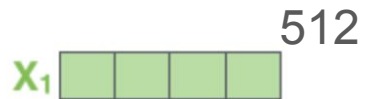
The self-attention mechanism computes a weighted sum of the input embeddings

Input

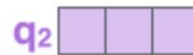
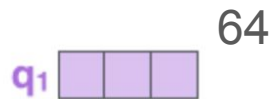
Thinking

Machines

Embedding

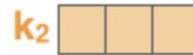
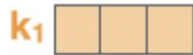


Queries



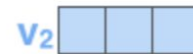
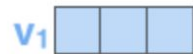
W^Q

Keys



W^K

Values



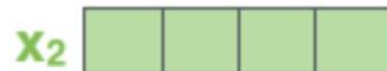
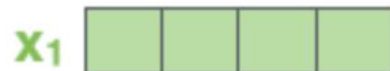
W^V

Input

Thinking

Machines

Embedding



Queries



Keys



Values



Score

$$q_1 \cdot k_1 = 112$$

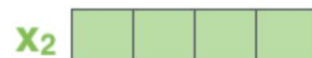
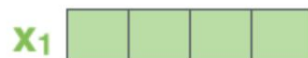
$$q_1 \cdot k_2 = 96$$

Input

Thinking

Machines

Embedding



Queries



Keys



Values



Score

$$q_1 \cdot k_1 = 112$$

$$q_1 \cdot k_2 = 96$$

Divide by 8 ($\sqrt{d_k}$)

14

12

Softmax

0.88

0.12

Why do we divide by $\sqrt{d_k}$ in attention?

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

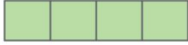
- Dividing by $\sqrt{d_k}$ keeps attention scores stable and learning smooth.
 - Dot products grow with vector size
 - It rescales the dot products
- Keeps numbers small and comparable

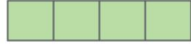
Input

Thinking

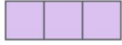
Machines

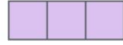
Embedding

x_1 

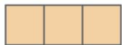
x_2 

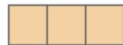
Queries

q_1 

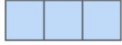
q_2 

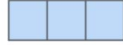
Keys

k_1 

k_2 

Values

v_1 

v_2 

Score

$q_1 \cdot k_1 = 112$

$q_2 \cdot k_2 = 96$

Divide by 8 ($\sqrt{d_k}$)

14

12

Softmax

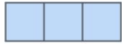
0.88

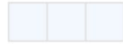
0.12

Softmax

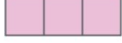
X

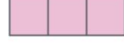
Value

v_1 

v_2 

Sum

z_1 

z_2 



$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

Q K^T V

A 2x4 purple grid labeled Q is multiplied by a 4x2 orange grid labeled K^T to produce a 2x2 pink grid labeled Z . The result Z is then used with a 2x4 blue grid labeled V to produce the final output.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Self-Attention Summary

- 1 Each word embedding $\rightarrow$ Query (Q), Key (K), Value (V)
- 2 Compute attention scores with dot product(Q, K)
- 3 Normalize with softmax $\rightarrow$ attention weights
- 4 Multiply weights $\times$ V $\rightarrow$ new contextualized representation

Multi-head Attention

1) This is our
input sentence*

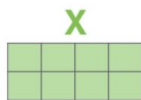
2) We embed
each word*

3) Split into 8 heads.
We multiply X or
 R with weight matrices

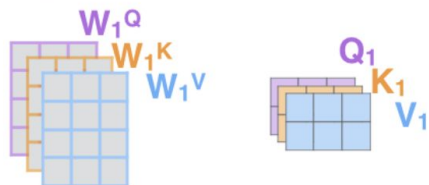
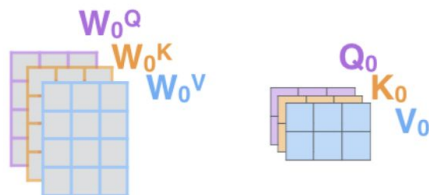
4) Calculate attention
using the resulting
 $Q/K/V$ matrices

5) Concatenate the resulting Z matrices,
then multiply with weight matrix W^O to
produce the output of the layer

Thinking
Machines



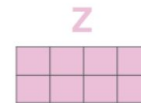
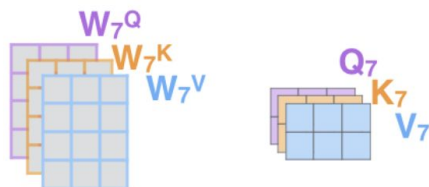
* In all encoders other than #0,
we don't need embedding.
We start directly with the output
of the encoder right below this one



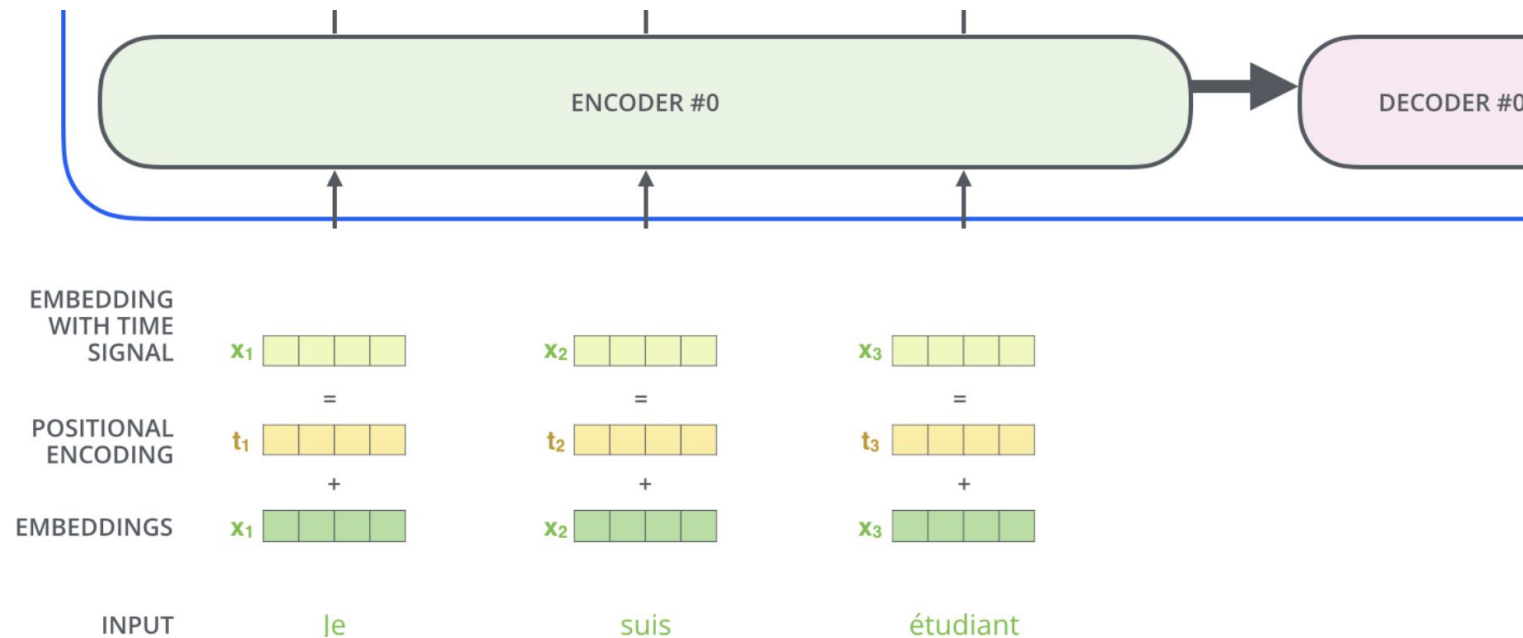
...

...

...

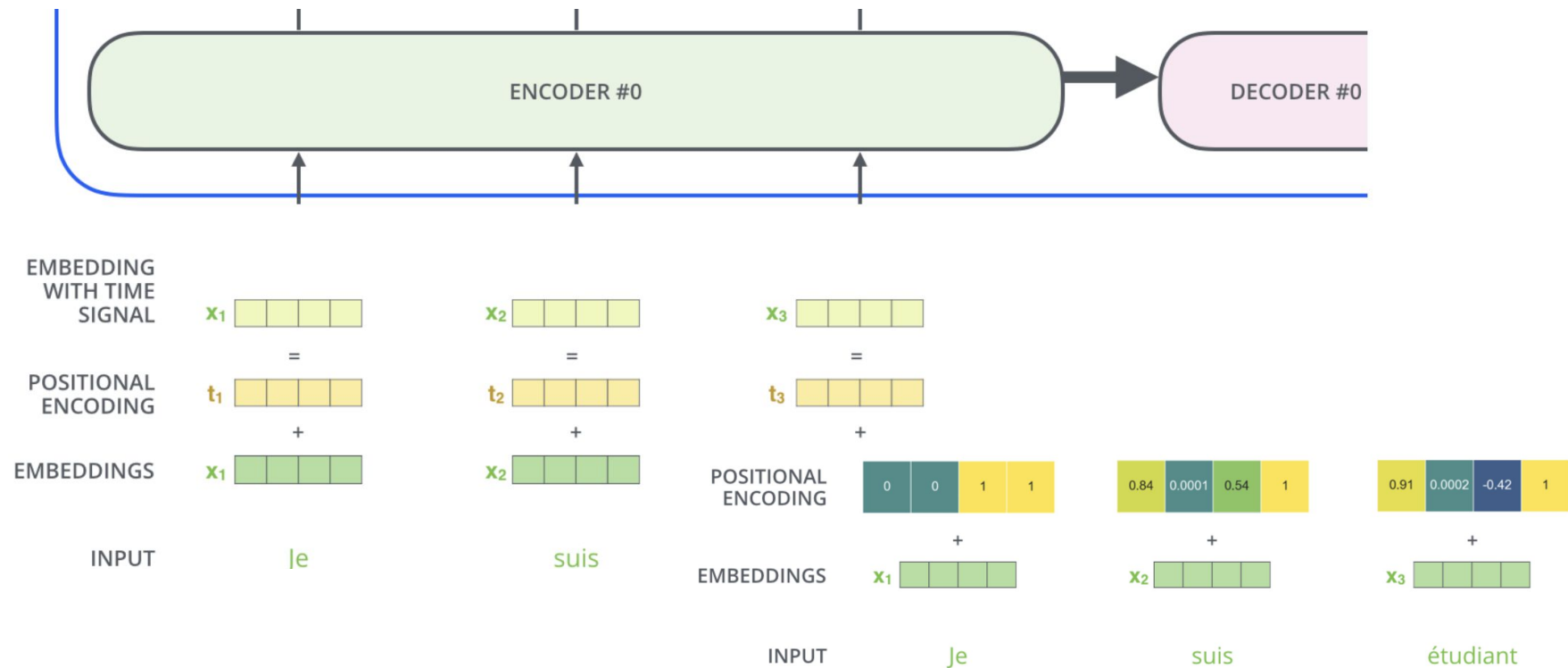


Positional Encoding



RNNs handle order through recurrence; Transformers need to inject order explicitly.

Positional Encoding



Positional Encoding

Fixed (sinusoidal) positional encoding

Introduced in the original Vaswani et al. (2017) paper

- Each position in the sequence is assigned a deterministic vector using sine and cosine functions.
- Not learned, fixed mathematical function.
- Provides smooth encoding of positions; nearby positions have similar representations.
- Allows extrapolation to longer sequences (beyond training length).

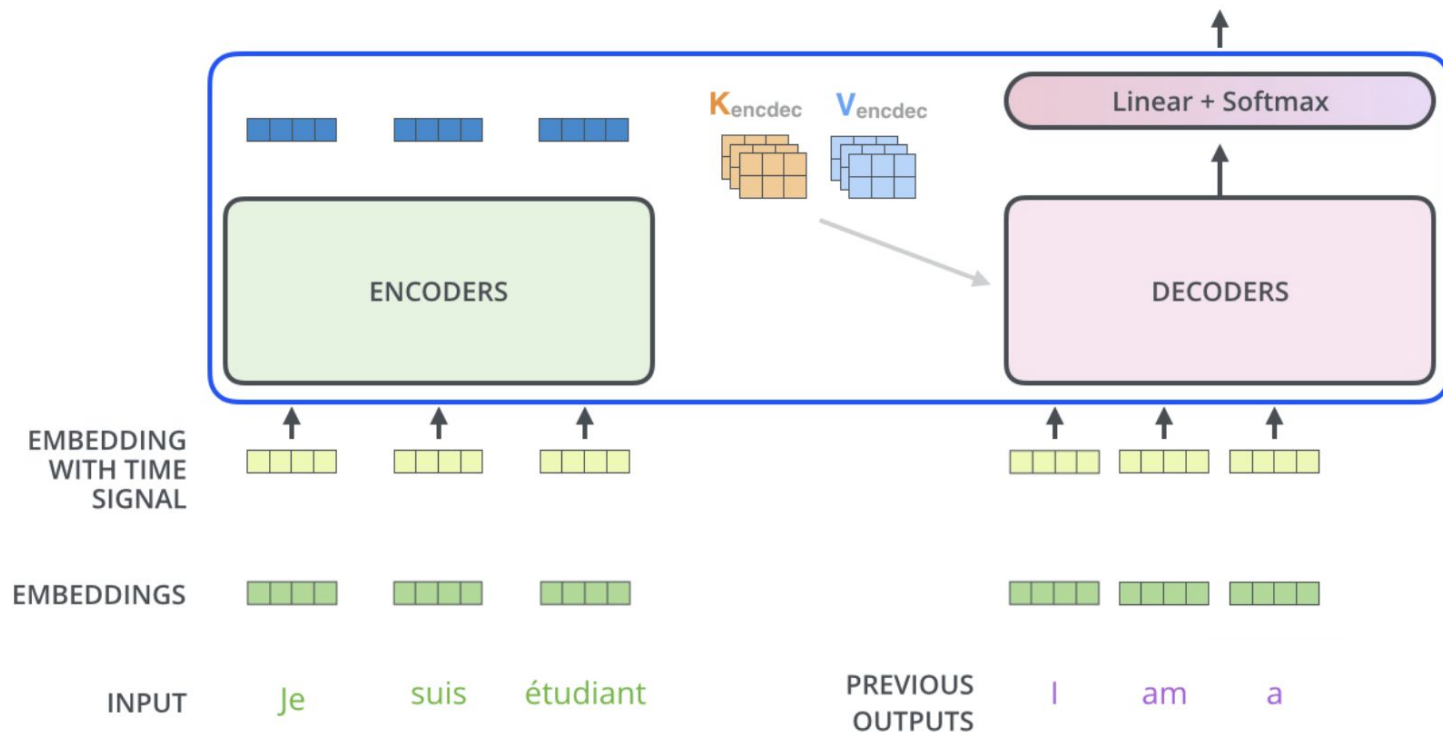
Learned positional embeddings

Each position gets its own trainable vector, just like a word embedding:

$PE_{pos} = \text{Embedding}(pos)$

- Learned jointly with the model parameters.
Typically performs slightly better in many NLP tasks (BERT, GPT).
- Cannot extrapolate to unseen sequence lengths (limited to training max length).
- BERT, GPT-2/3, T5

The Decoder

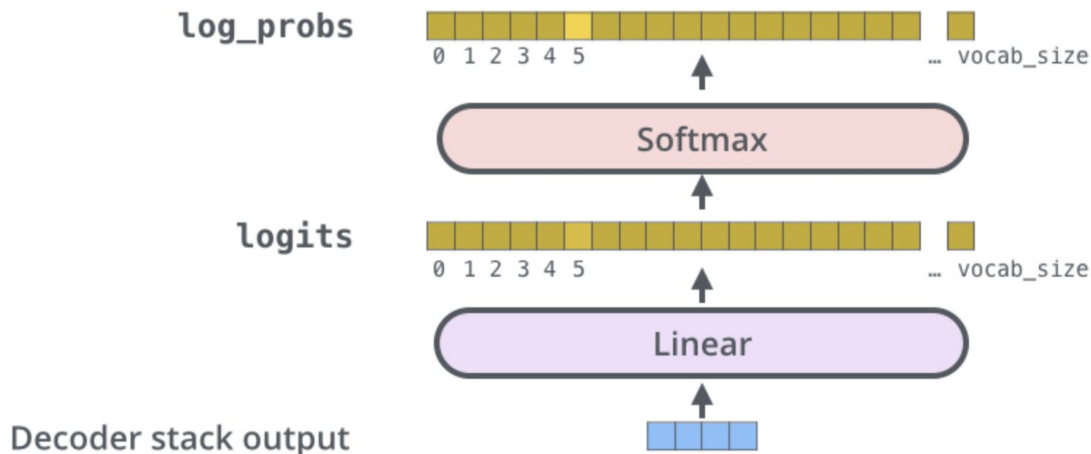


Linear Layer and Softmax

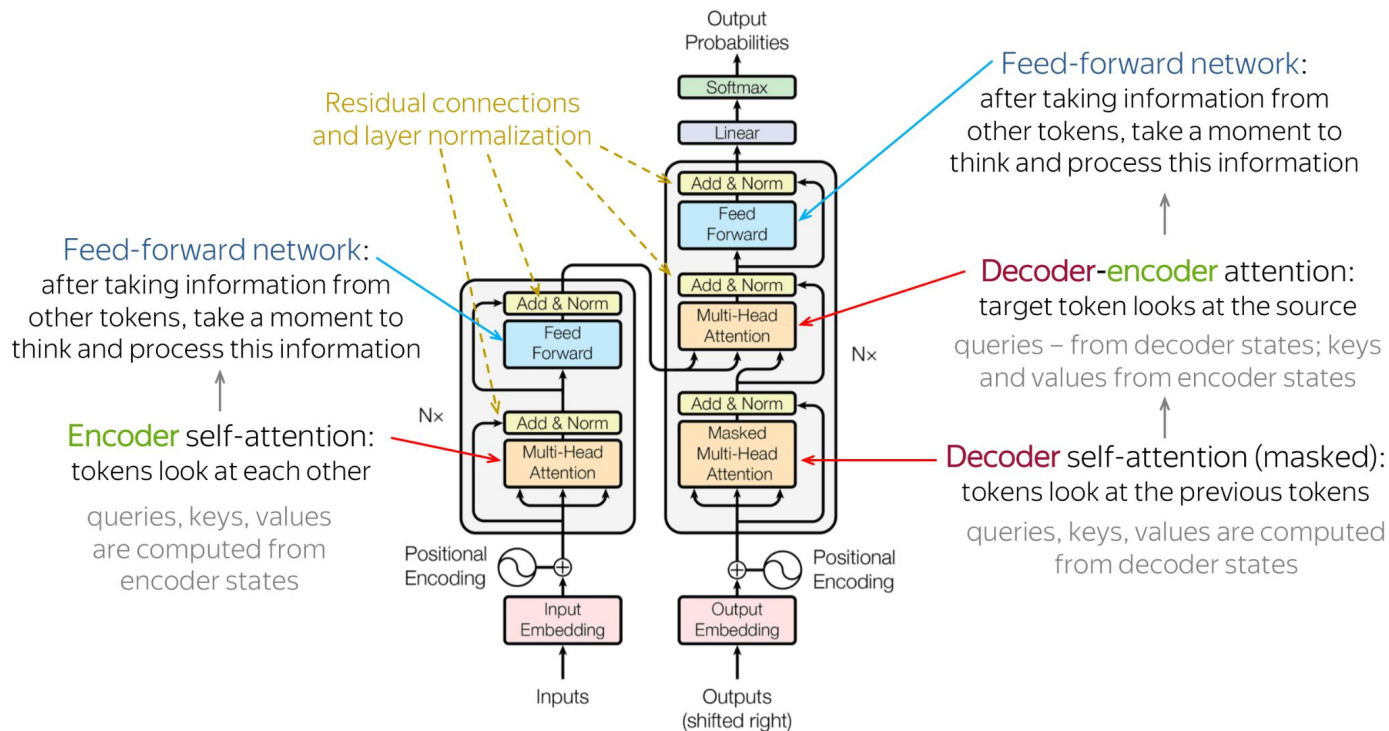
Which word in our vocabulary
is associated with this index?

am

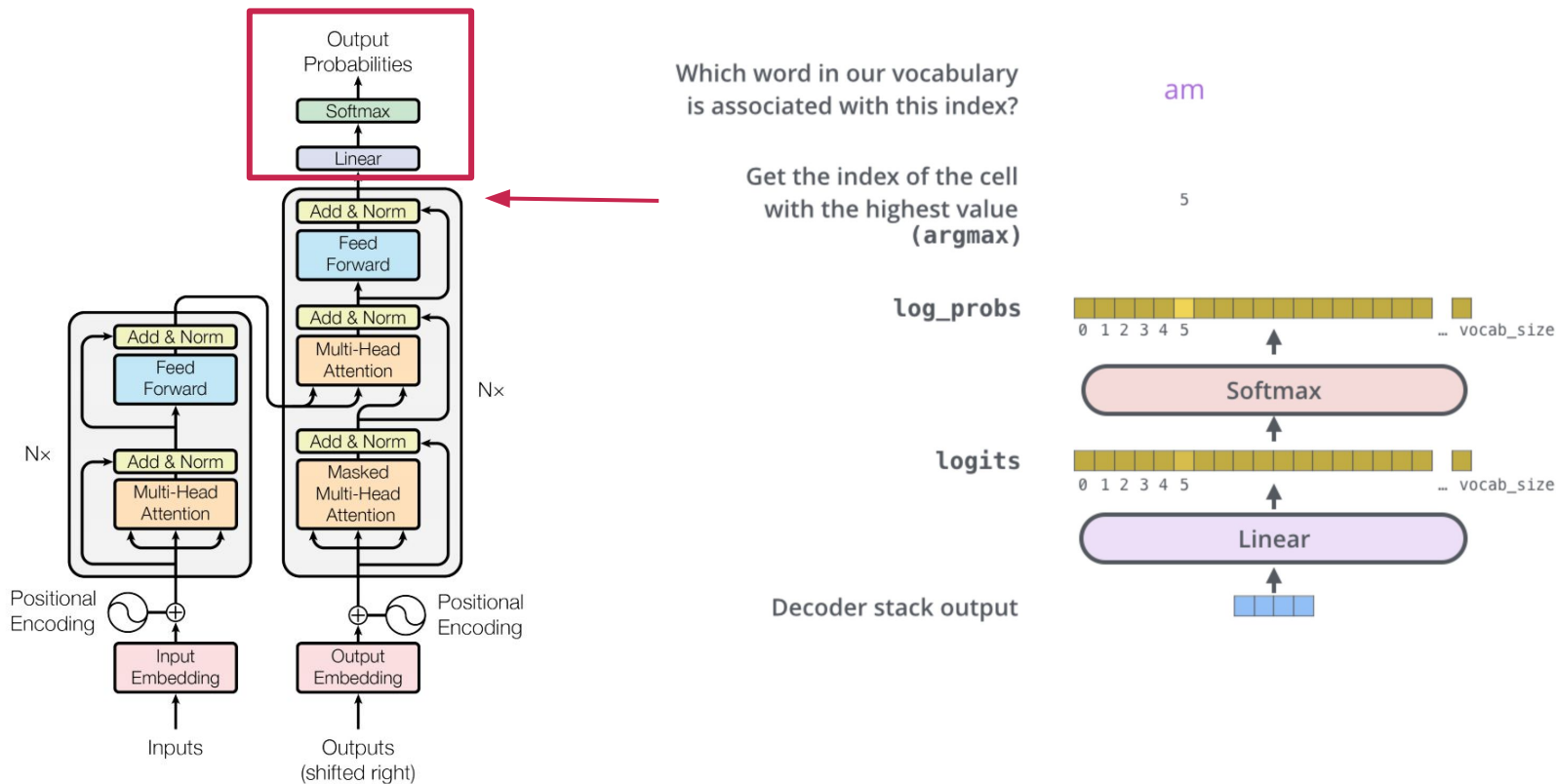
Get the index of the cell
with the highest value
(argmax)



Transformers (Vaswani et al., 2017)

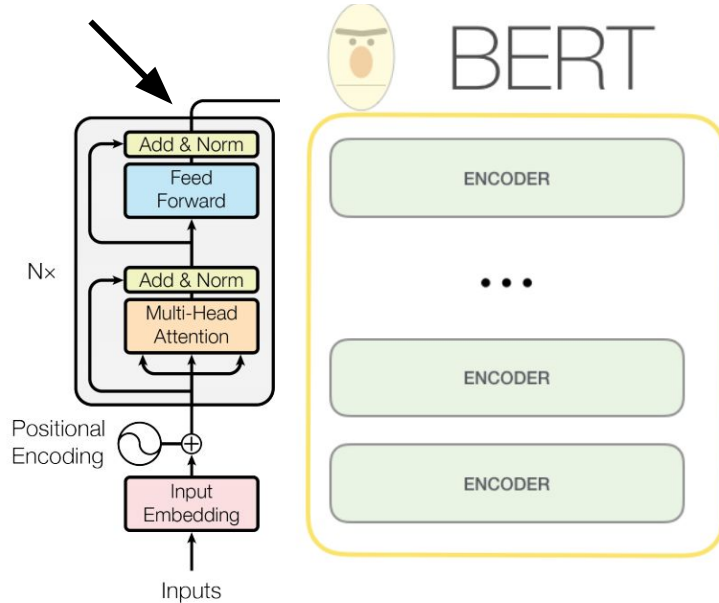


Transformers (Vaswani et al., 2017)



Transformers for Language Modelling

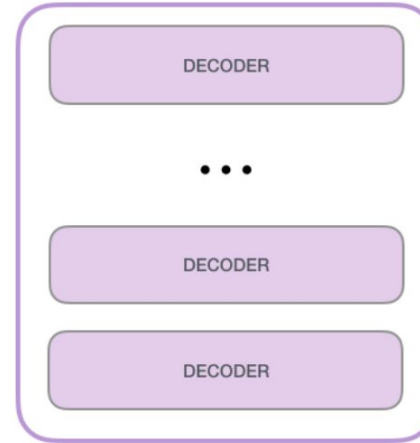
ENCODER



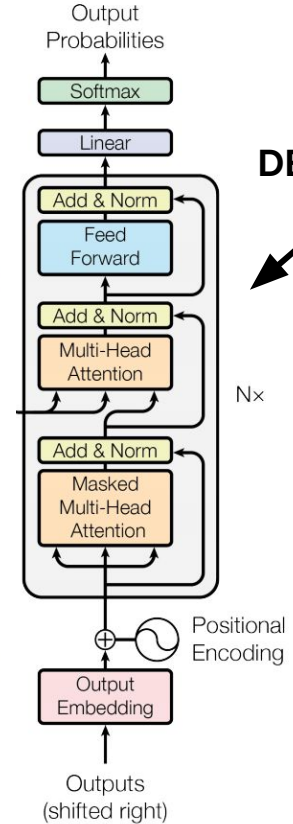
BERT



GPT-2

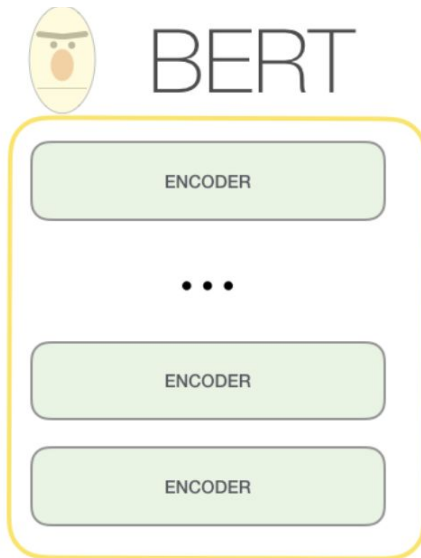
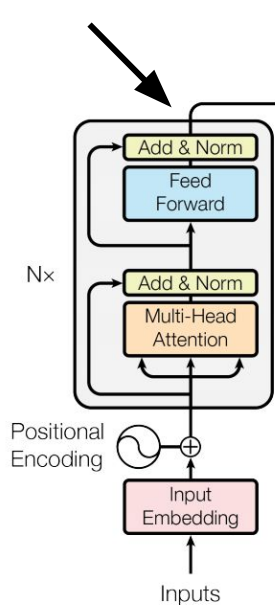


DECODER



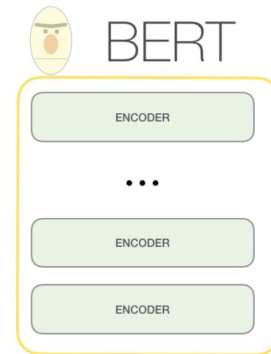
Bidirectional Transformer – BERT

ENCODER



S35

Bidirectional Encoder Representations from Transformers (BERT)



From Static to Contextual Word Embeddings

Static (Word2Vec, GloVe)

One embedding per word

“Bank” → same vector

Trained on local co-occurrences

Contextual (BERT)

Embedding depends on sentence context

“Bank” in *river bank* ≠ “Bank” in *bank account*

Trained with deep bidirectional context

But how do we get these contextual embeddings?

We need to process the entire sentence (encoder), not predict the next word.

From words to tokens

Words (BoW, Word Embeddings)

- Text is split into words
- Each word $\rightarrow$ one vector
- Fixed vocabulary

Problems

- Unknown words (OOV)
- Large vocabularies
- Morphology explodes space
(*run, runs, running, runner*)

From words to tokens

A token is a unit decided by a tokenizer

- Token length is **learned from data**.

Tokens can be:

- whole words
- parts of words
- punctuation or symbols

"playing football"

["play", "ing", " football"]

index: [5342, 91, 2087]

From words to tokens

1. Start with characters
2. Find frequent character sequences
3. Merge them into tokens
4. Stop when vocab size is reached (e.g. 30k)

- Frequent words → **1 token**
- Rare words → **multiple tokens**

Balances:

- vocabulary size
- flexibility
- robustness to unseen text

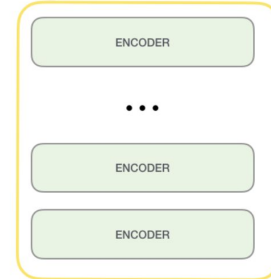
```
English tokens: ['The',  
'model', 'learns',  
'representation',  
'##s', 'from', 'text',  
'.']
```

English #tokens: 8

```
Danish tokens:  
['Modelle', '##n',  
'lærer', 'rep', '##r',  
'##æ', '##sent',  
'##ation', '##er',  
'fra', 'tekst', '.']
```

Danish #tokens: 12

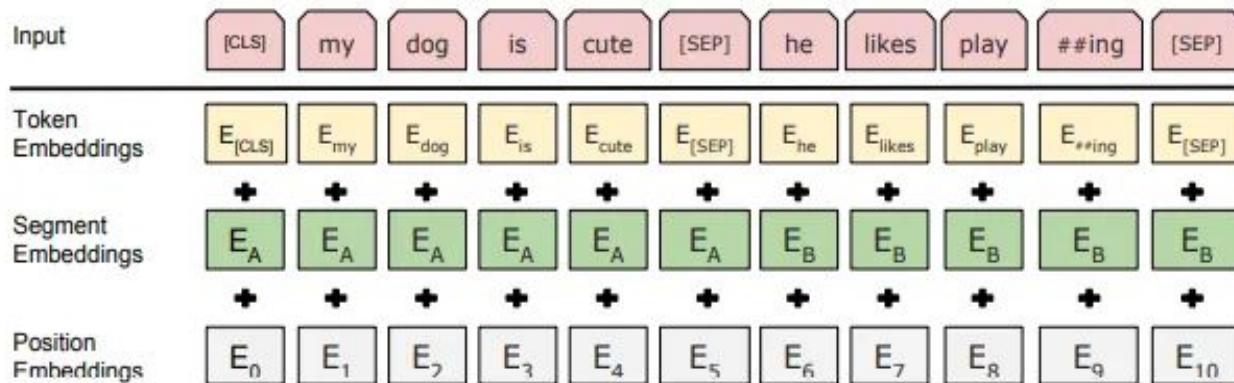
Bidirectional Encoder Representations from Transformers (BERT)



[CLS] stands for classification tasks and it aims to **summarize the sentence** and the output of the last layer of this token

Each **token** has **direct access (attention) to all tokens**.

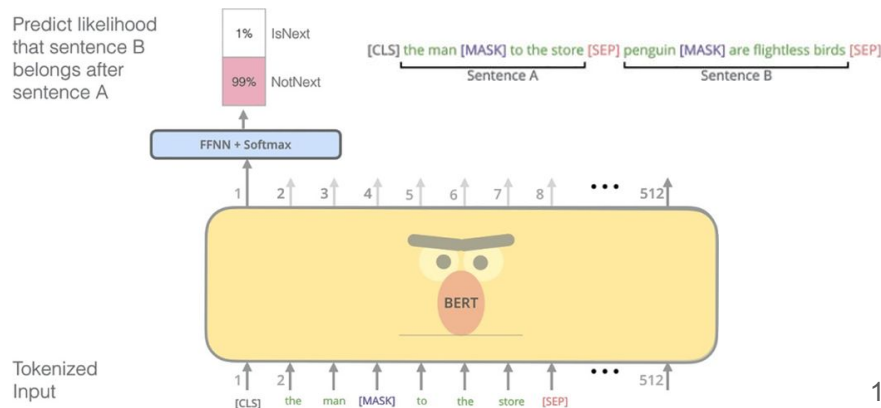
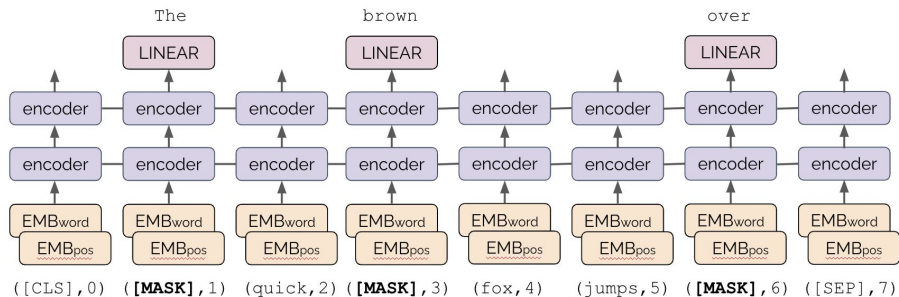
The **[SEP]** token indicates when the next sentence starts.



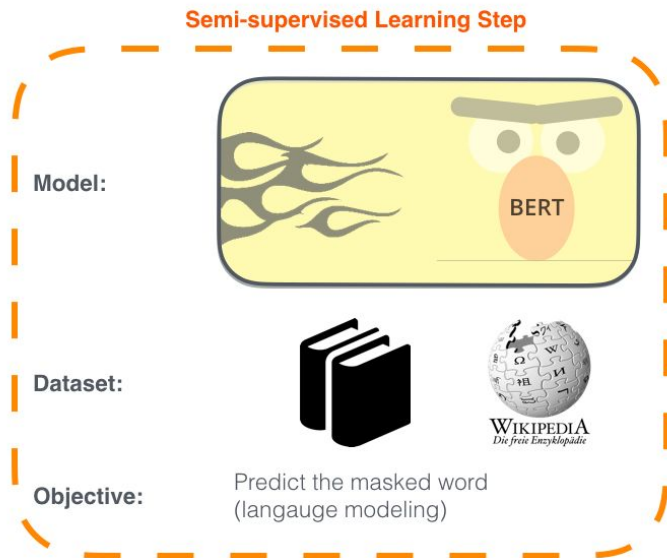
BERT

- **Masked Language Modelling (MLM):** Mask a subset of tokens, and try to predict them.
- **15%** of the tokens are **masked**.

- **Next Sentence Prediction (NSP):** Provide a **pair of sentences**, and **predict** if the **second sentence follows the first** in the original document.

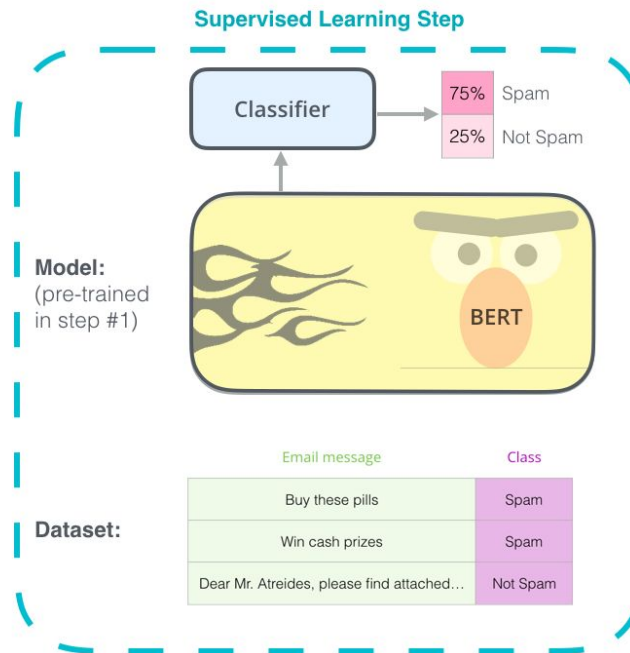


Pre-training → Fine-tuning



Pre-training Corpora:

- English **Wikipedia** Articles
- Toronto **Books** Corpus (TBC)



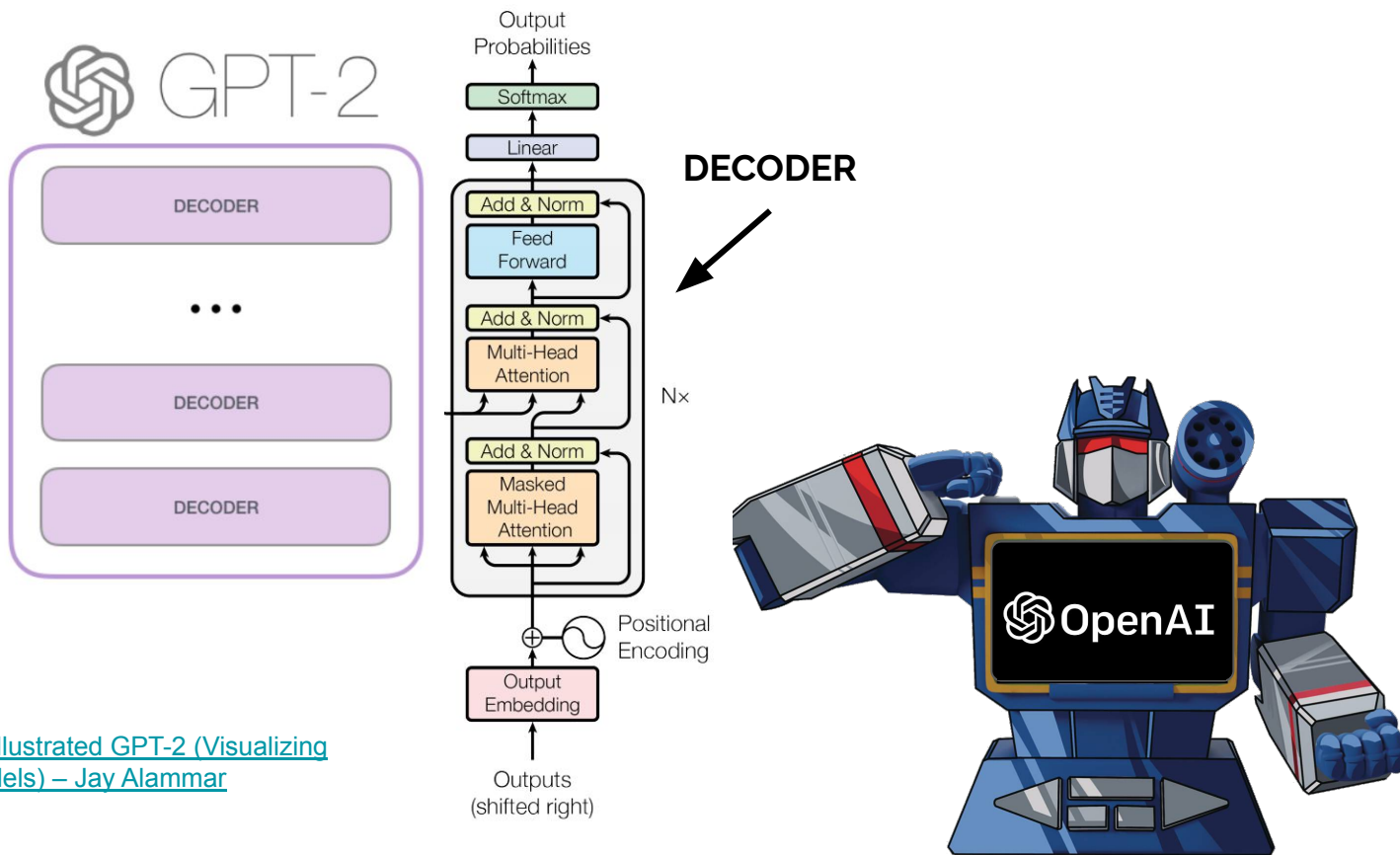
Fine-tuning:

- Task specific data
- A fine-tuned model for each task

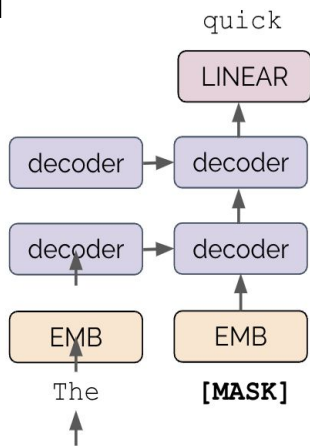
BERT vs GPT

- Reads the entire sentence at once (bidirectional).
 - Learns contextual representations for understanding.
 - Trained with Masked Language Modeling (MLM): fill in missing words.
 - Great for:
 - Sentence classification
 - Question answering
- Learns to **predict the next word**
 - Trained with **causal (autoregressive) language modeling**:
 $P(w_t | w_1, w_2, \dots, w_{t-1})$
 - Great for:
 - Text generation
 - Dialogue systems
 - Story completion, code generation, etc.

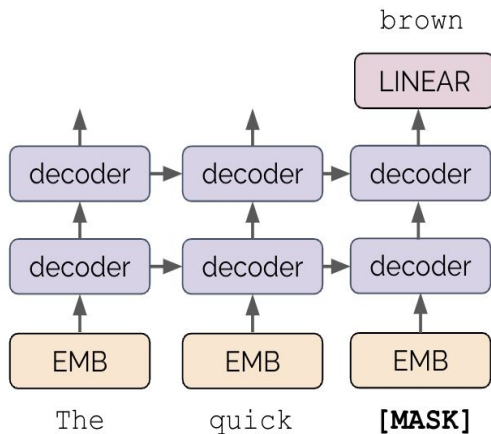
Generative Pre-Training - GPT



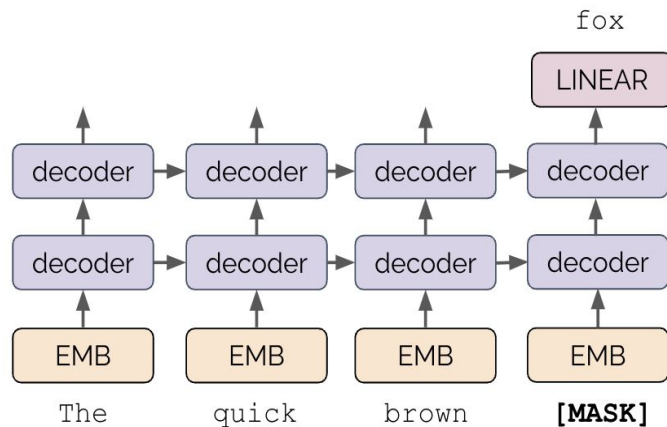
GPT



The model computes $P(\text{next word} | \text{previous words})$ for each step, and we minimize $-\log P(\text{correct word})$ over all tokens.



117 M parameters, trained on a 40GB dataset (WebText)



What happens during large-scale training

We train over *massive corpora*

Each example is just a chunk of text, and the model repeatedly predicts the next token

The model learns:

- Grammar and syntax (how sentences flow)
- World knowledge (co-occurrence of entities, facts)
- Discourse structure (how ideas connect over long text)

Cross-entropy loss

$$\mathcal{L} = - \sum_{t=1}^n \log P(w_t | w_1, \dots, w_{t-1})$$

Each wrong prediction increases the loss, so the model updates its weights to assign higher probability to the correct next token next time.

Trained on raw text without labels,
Self-supervised task: the “label” for each input is the next token.

Decoding Strategies

Decoding strategies guide how generative models (e.g., GPT) select the next token in a sequence. These strategies balance **coherence** (logical, meaningful outputs) with **variability** (diverse, creative outputs).

- Greedy Decoding
 - Always selects **the token with the highest probability** at each step.
- Temperature
 - A scaling factor that **adjusts the sharpness** of the probability distribution.
- Top-k Sampling
 - Restricts token selection to the **k most probable tokens**.

Decoding Strategies: Greedy Decoding

Always selects the token with the highest probability at each step.

- Picks the most probable token without randomness.
- Output is deterministic and coherent but may lack diversity.

Example:

The sky is ____

Output: blue

Decoding Strategies: Temperature

A scaling factor that adjusts the sharpness of the probability distribution.

- Sharper distribution: reduces variability (more deterministic).
- Flatter distribution: increases variability (more creative).

Example:

The sky is ----

Sharper: blue

Flatter: pinkish-gray / infinite

$$P_T(y_i) = \frac{e^{(z_i/T)}}{\sum_j e^{(z_j/T)}}$$

Decoding Strategies: Top-k Sampling

Restricts token selection to the k most probable tokens.

- Sort tokens by probability.
- Retain the top tokens and sample from them.

Example:

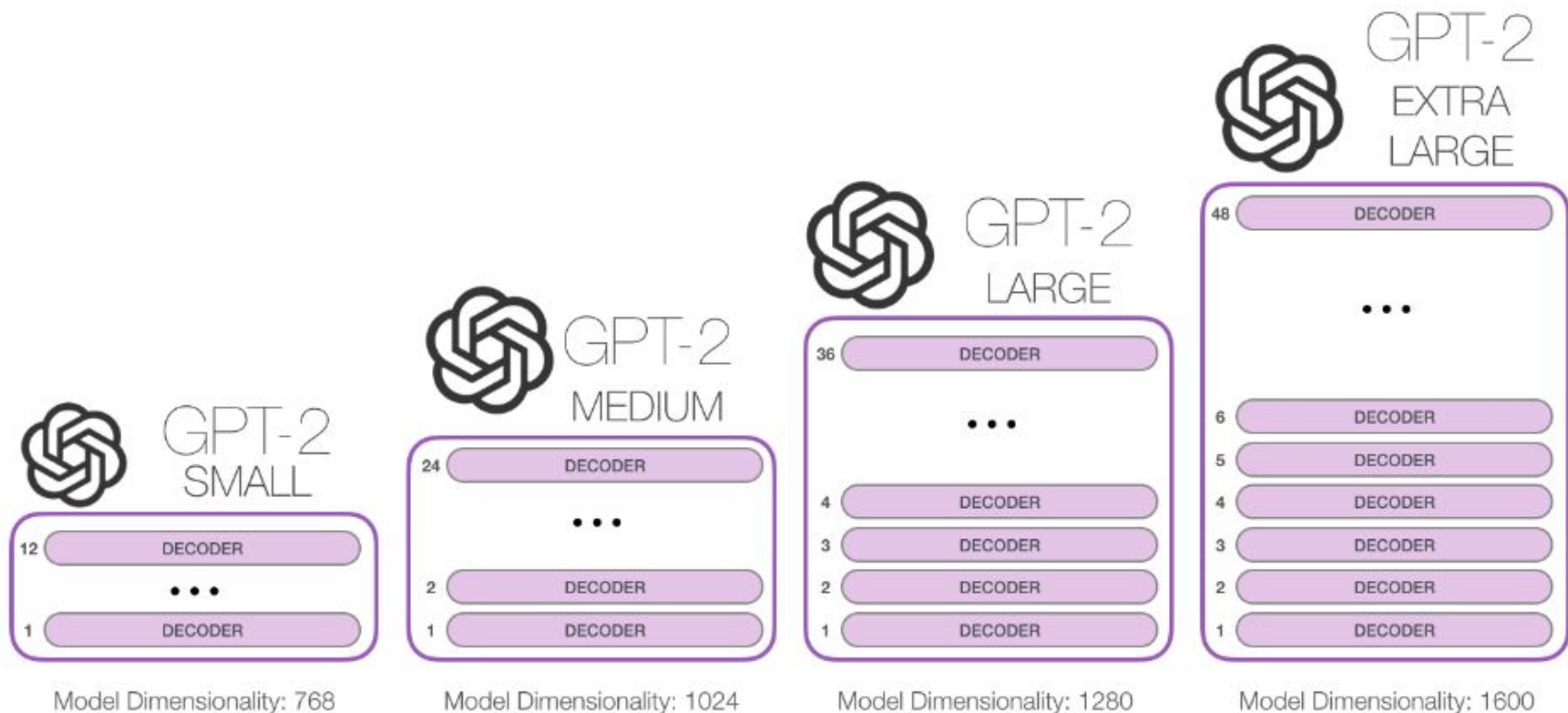
The sky is ____

Output: blue, clear, bright, infinite

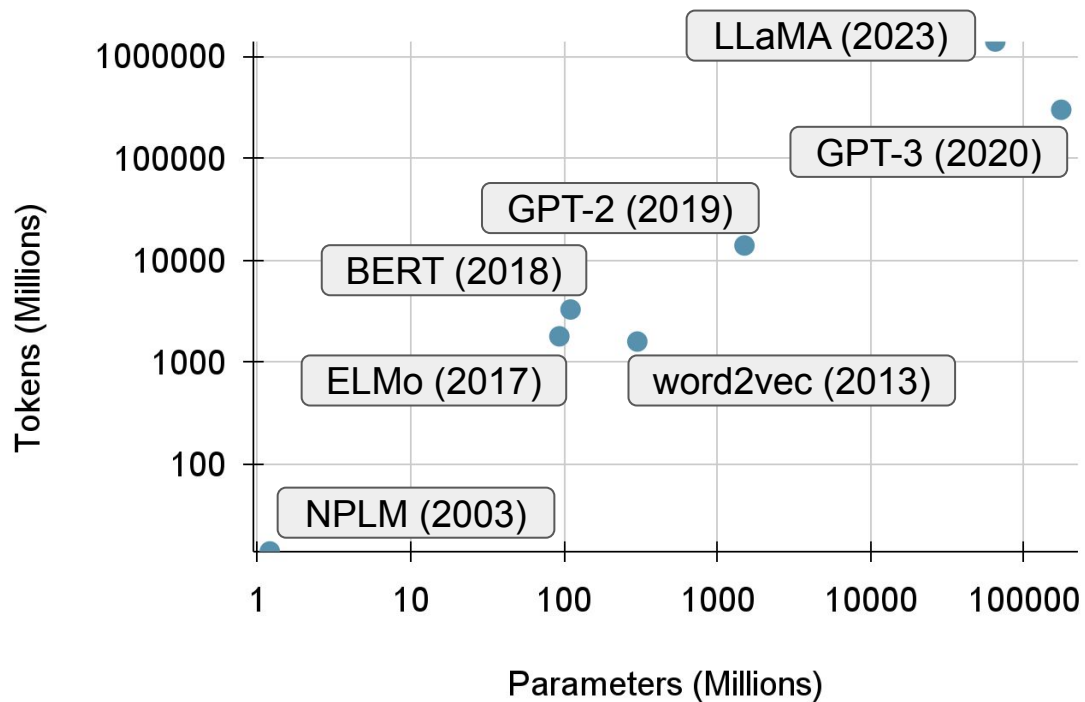
Comparing Strategies

Method	Parameter	Effect on Variability	Example Cases
Greedy Decoding	None	No variability, deterministic.	Best for predictable, deterministic tasks.
Temperature	(e.g., 0.7)	Adjusts randomness across the entire distribution.	High Variability: creative writing, brainstorming. Low Variability: Summarization, factual outputs.
Top-k Sampling	(e.g., 10)	Limits choices to a fixed number of tokens.	Fixed token limits; useful for tasks requiring bounded creativity.

GPT-2 Language Models are Unsupervised Multitask Learners (Radford et al., 2018)



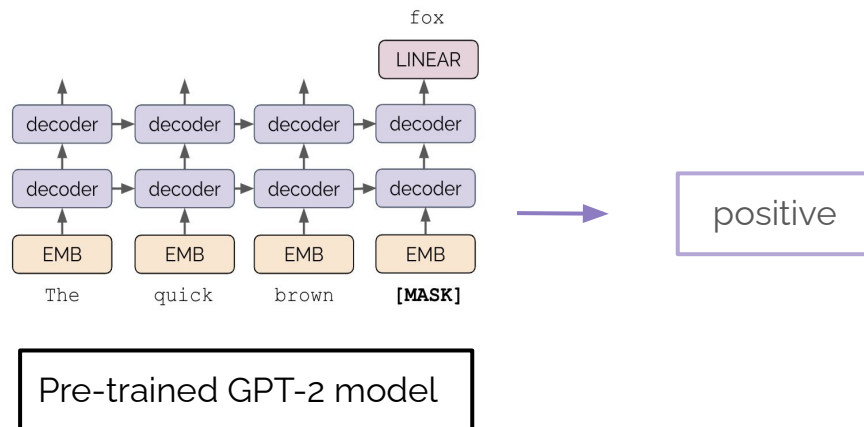
LLMs



Zero-shot Prompting

Any given task is a seq2seq (generational) task! (Shchick & Schutze 2020)

Movie Review: "Yesterday, I watch Jaws. It was quite interesting, I think it definitely worth-watching!"
Is this review (a) positive or (b) negative? _____



Zero-Shot with GPT-2

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer

# Load pre-trained GPT-2 model and tokenizer
model_name = "gpt2"
model = GPT2LMHeadModel.from_pretrained(model_name)
tokenizer = GPT2Tokenizer.from_pretrained(model_name)

# Define a prompt for zero-shot learning
prompt = "Translate the following sentence to French: 'The cat is on the  
roof.'\nTranslation:"

# Tokenize the input prompt
inputs = tokenizer(prompt, return_tensors="pt")

# Generate the response
outputs = model.generate(**inputs, max_length=50, num_return_sequences=1)

# Decode the generated text
generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)

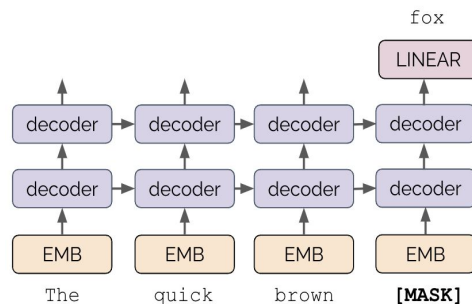
print(generated_text)
```

Zero-shot Prompting

Variability in Prompts: the choice of prompt is very important for zero-shot settings!

Movie Review: "Yesterday, I watch Jaws. It was quite interesting, I think it definitely worth-watching!"

Is this review positive or negative?
Review: "Yesterday, I watch Jaws. It was quite interesting, I think it definitely worth-watching!" **Is this positive, negative? // _____**



Pre-trained GPT-2 model

positive

negative

Few-shot Prompting

Form “demonstration examples” from (x, y) pairs

Movie Review: “The cinematography was stellar; great movie!” Is this review (a) positive or (b) negative? Positive

Movie Review: “The plot was boring and the visuals were subpar.” Is this review (a) positive or (b) negative? Negative

Movie Review: “The movie’s acting could’ve been better, but the visuals and directing were top-notch.” Is this review (a) positive or (b) negative? Positive

Movie Review: “Yesterday, I watch Jaws. It was quite interesting, I think it definitely worth-watching!” Is this review (a) positive or (b) negative? _____

Few-Shot with GPT-2

```
# Define a prompt with a few-shot learning approach
prompt = (
    "Translate the following sentences to French:\n"
    "1. 'The cat is on the roof.'\n"
    "Translation: 'Le chat est sur le toit.'\n\n"
    "2. 'The dog is in the garden.'\n"
    "Translation: 'Le chien est dans le jardin.'\n\n"
    "3. 'The bird is in the tree.'\n"
    "Translation: 'L'oiseau est dans l'arbre.'\n\n"
    "4. 'The fish is in the pond.'\n"
    "Translation: "
)

# Tokenize the input prompt
inputs = tokenizer(prompt, return_tensors="pt")

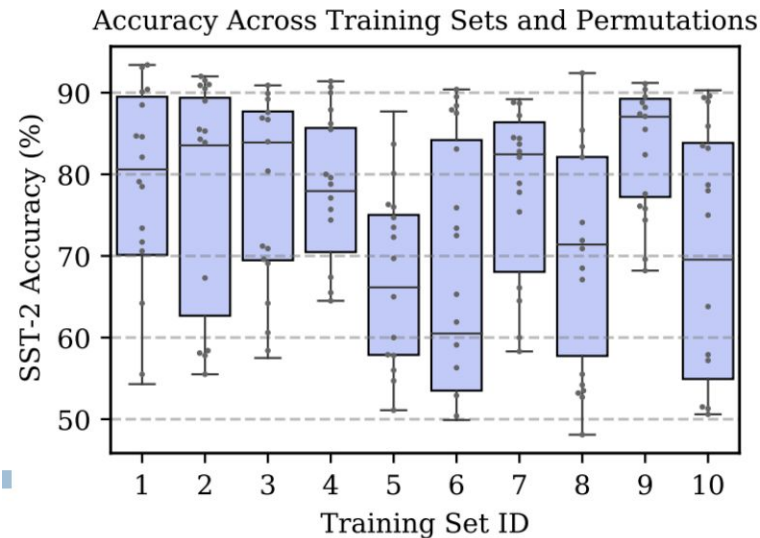
# Generate the response
outputs = model.generate(**inputs, max_length=100, num_return_sequences=1)

# Decode the generated text
generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)

print(generated_text)
```

Few-shot Prompting

- Few-shot Prompting does not require parameter updates and is directly performed on pretrained LLMs.
- The performance strongly relies on the demonstration surface, including demonstration format, the order of demonstration examples, and so on (Zhao et al., 2021).



[Zhao et al., \(2021\) Calibrate Before Use Links to an external site.](#)

Instruction-based fine-tuning

We want to optimize models for $P(\text{answer}|\text{prompt, input})$, but they only learned a basic language modeling objective (e.g., **predicting the next word**)

We could treat the basic language modeling as pre-training, then fine-tune them on what we care about.

Pre-training → **Fine-tuning** on task specific data

Instruction-based fine-tuning

Instruction tuning: supervised fine-tuning on data derived from **many NLP tasks described via instructions.**

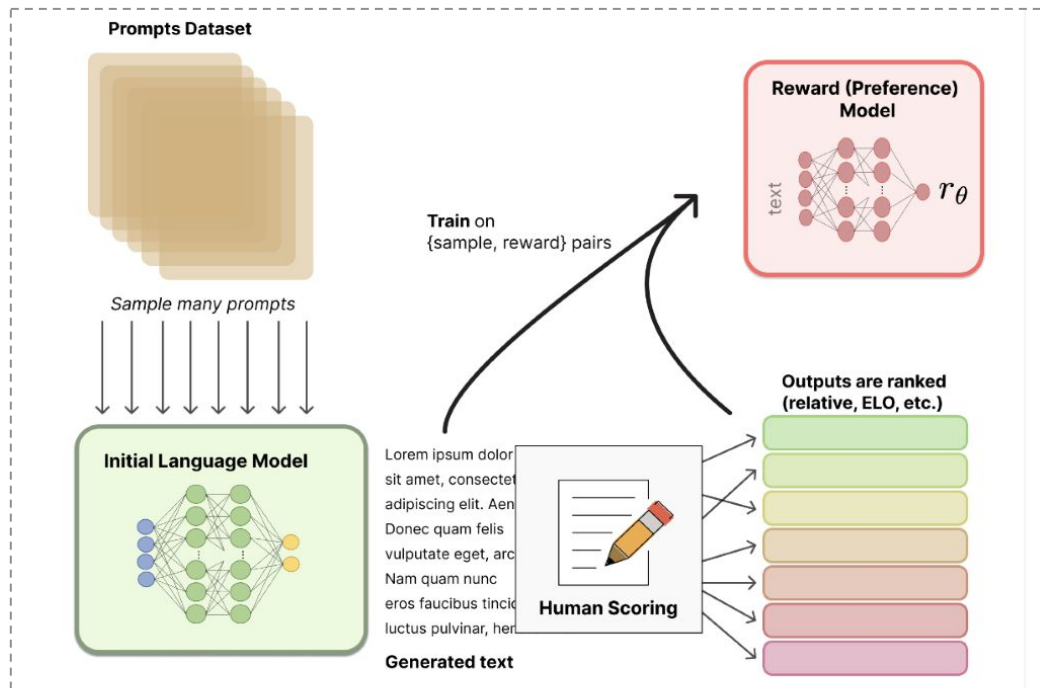
Instruction	Input	Output
“Translate to French”	“Good morning”	“Bonjour”
“Classify the sentiment”	“I loved the movie!”	“Positive”

So the model learns: *When I see a text that looks like an instruction, I should follow it.*

Reinforcement Learning from Human Feedback (RLHF)

Alignment with Human Feedback -
RLHF (Ouyang et al., 2022)

- Aligning models' responses with human desires leads to better responses.



Reinforcement Learning from Human Feedback (RLHF)

1. **Supervised fine-tuning (SFT)**

Start with an instruction-tuned model.

Train it on examples of good instruction–response pairs written by humans.

2. **Reward model (RM)**

Show human annotators multiple model answers to the same prompt.

They rank which answer they prefer.

The model learns to predict this *reward signal*.

3. **Reinforcement learning (RL)**

Use the reward model to fine-tune the language model so it produces responses that maximize human preference scores.

Reinforcement Learning from Human Feedback (RLHF)

Pre-training

Step 0.1

Collect large text corpora and train a GPT model.

A document is sampled from our corpus.

Barack Obama
was born in
August 4, 1961.

The document is used to train a GPT model with self-supervision.



Barack Obama
was born ...

Instruction-tuning

Step 0.2

Fine-tune the pre-trained GPT model on instruction-based datasets.

A document is sampled from the datasets.

When was Barack
Obama born?
In August 4, 1961.

The document is used to train a GPT model with self-supervision.



When was Barack
Obama born?

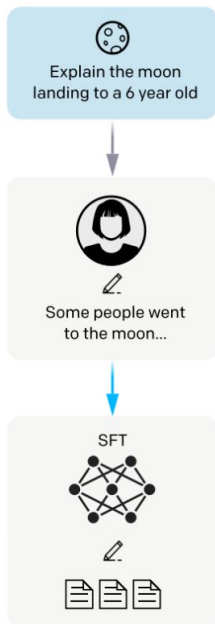
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is
sampled from our
prompt dataset.

A labeler
demonstrates the
desired output
behavior.

This data is used
to fine-tune GPT-3
with supervised
learning.



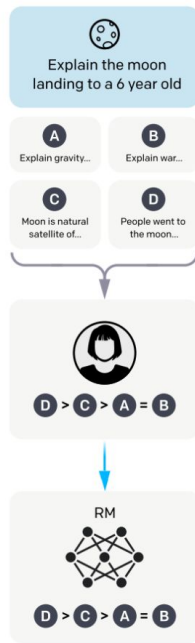
Step 2

Collect comparison data, and train a reward model.

A prompt and
several model
outputs are
sampled.

A labeler ranks
the outputs from
best to worst.

This data is used
to train our
reward model.



Step 3

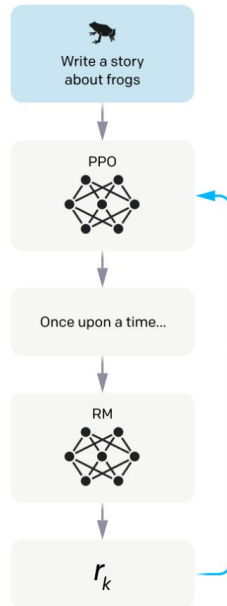
Optimize a policy against the reward model using reinforcement learning.

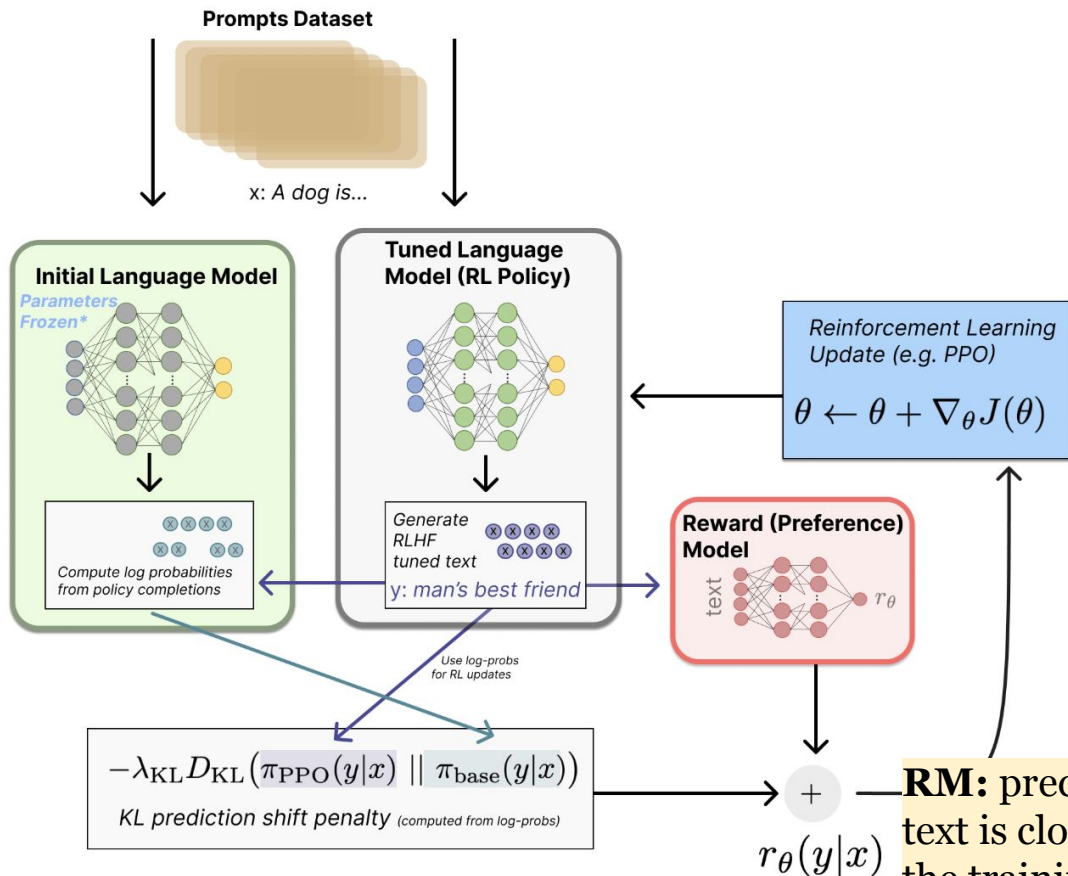
A new prompt
is sampled from
the dataset.

The policy
generates
an output.

The reward model
calculates a
reward for
the output.

The reward is
used to update
the policy
using PPO.





Reinforcement Learning (RL) Training Loop

- The policy model generates responses to training prompts
- A reward model evaluates and scores these responses
- An RL optimizer updates the model's weights based on the reward signal
- This process is repeated iteratively to gradually improve the policy

RM: predicts the probability that a piece of text is close to a “preferred” piece of text from the training comparisons.

Reinforcement Learning from Human Feedback (RLHF)

Table 1: Distribution of use case categories from our API prompt dataset.

Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

Table 2: Illustrative prompts from our API prompt dataset. These are fictional examples inspired by real usage—see more examples in Appendix A.2.1.

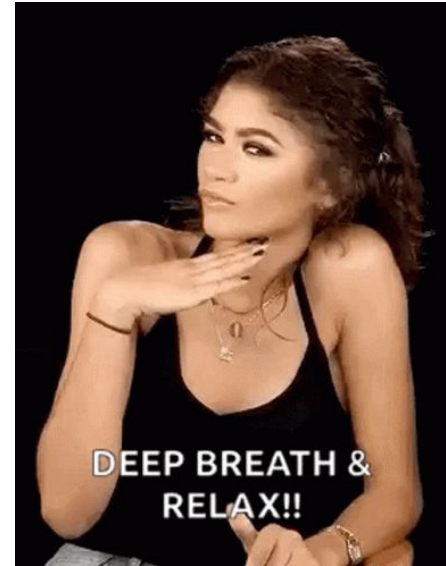
Use-case	Prompt
Brainstorming	List five ideas for how to regain enthusiasm for my career
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.
Rewrite	This is the summary of a Broadway play: "" { summary } "" This is the outline of the commercial for that play: ""

4. Limitations of Large Language Models

Limitations

Understanding Limitations is Crucial

- Ensuring realistic expectations
- Identifying areas for improvement
- Mitigating risks and ethical concerns



Chain-of-Thought (CoT) Reasoning

- A prompting technique where the model explicitly generates intermediate reasoning steps before the final answer.
- Encourages “step-by-step” reasoning similar to how humans solve problems.

Q: The rope was 10m long and was cut in half. How long is one piece?

A: Let's think step by step.

1. The rope is 10m.

2. Cutting in half divides it into two equal parts.

3. $10 \div 2 = 5$.

Answer: 5m

Chain-of-Thought (CoT) Reasoning

- Improves performance on reasoning-intensive tasks (math, logic, QA).
- Makes reasoning processes interpretable.
- But: reasoning can be post-hoc (steps generated after predicting, to justify output).

Example:

Step 1: The capital of France is Rome.

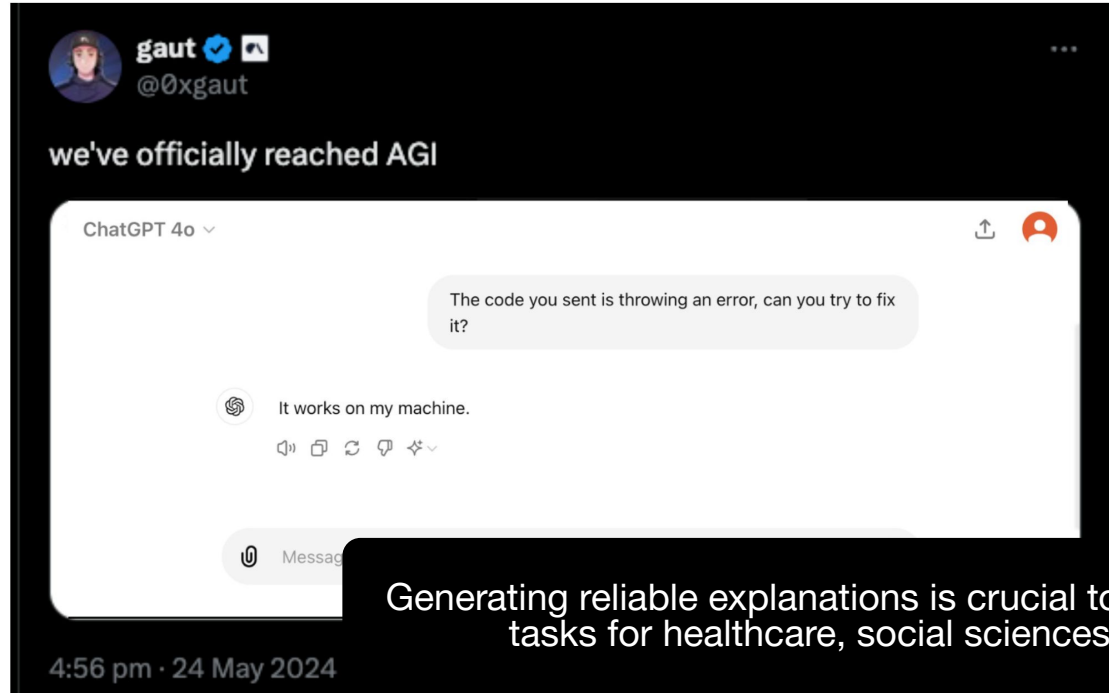
Step 2: Rome is in Italy.

Step 3: So the answer is Italy.

The reasoning sounds structured, but it's *wrong at step 1*

The model never “realized” its first mistake, so the reasoning just justifies a wrong intuition.

Generating Reliable Explanations



Limited Language Coverage

NLP usually covers **100 languages out of 3,000 written languages** (400 with >1M speakers)

We can increase coverage by training on datasets of low-resource languages



Limited Multicultural Awareness

LLMs have limited understanding of multicultural concepts



Rethinking Vision and Language



Languages

- Mostly in English
- Or some Indo-European Languages



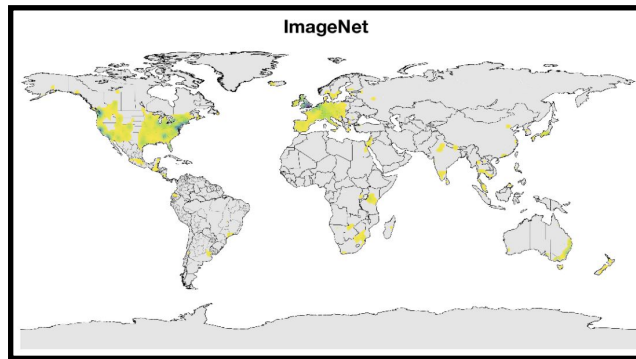
La danza de los viejitos

Image sources

- Mostly from ImageNet or COCO
- Reflecting North American and European cultures

Implications for V&L models

- Narrow linguistic/cultural domain
- No way to assess their real-world comprehension



Density map of geographical distribution of images in ImageNet (DeVries+, 2019)

Limited Multicultural Awareness

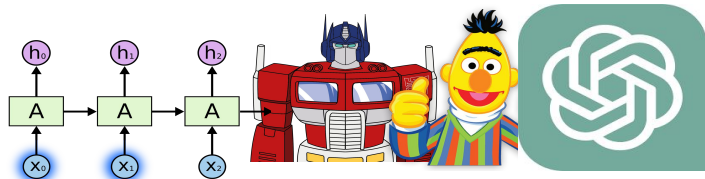
LLMs have limited understanding of multicultural concepts

Including native speakers into the data creation process can help to mitigate the underrepresentation of certain cultures and languages.



Summary

Neural Language Models



Multilinguality & Multicultural LLMs

